When Does a Reader-Side Lifetime Fix Help? Admission Coupling in an LSM Engine

V V Ramarathinam Iyer
Department of Database Systems, School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
Email: ram.iyer200722@gmail.com

***Abstract*—A read lock held across a lookup is an easy way to keep a storage file alive, and replacing it is standard advice, yet a reader-side fix cannot shorten the writer's own critical section. We ask when such a fix helps. In six pre-registered campaigns on one LSM engine, with read cost controlled by injected delay, we compare a coarse read lock, per-table leases, and immutable reader sets across write load, compaction, and table count. Readers and writers are coupled through lock admission: each exclusive acquisition in a commit admits a round of readers, so the coarse lock's read gain at short lookups is paid for in writes and did not appear at equal write rates. An intervention changing the number of acquisitions supported this. The writer also bounds what any reader-side design can deliver: exploratory runs at a high write rate showed the writer holding the lock across fsync for about three quarters of the time, with reads following the time left. Leases and reader sets matched on throughput within a declared margin until read-only load at 256 tables, where leases completed 34% fewer reads, and differed in commit tails under compaction. Practically, measure lock-held lookup time, write load, and table count before replacing a lock. Back-to-back trials carried an fsync stall of about 55 ms into the next trial, distorting tails while rechecked throughput classes held. Results come from one engine on one Windows host, at most one writer, injected delay, and WAL fsync under the exclusive lock.**

***Index Terms*—LSM tree, reader-writer lock, descriptor leasing, immutable snapshots, controlled experiment.**

I. Introduction

A reader-side lifetime fix shortens the reader's lock hold but leaves the writer's own critical section in place. An LSM engine buffers updates in memory and writes sorted runs to disk [1], [2]; readers need stable file references while flushes and compactions replace those runs. Holding the manager's read lock across a lookup provides that protection but makes exclusive acquisitions wait for the lookup, including file I/O. We ask: when does a reader-side lifetime fix actually help, given the writer's own critical section?

Production engines separate lifetime protection from long read holds. LevelDB, RocksDB, and Pebble retain references after releasing a short coordination lock [3]–[5]. We compare CoarseLock, which retains the manager lock, with per-table leases (Lease) and immutable reader sets (Snapshot). Injected delay controls lookup cost while the writer's validation and WAL synchronization remain shared across designs.

The paper makes four contributions:

- **Admission coupling.** Under a saturated, continually waiting writer, CoarseLock's reads couple to writes as $R \simeq k\, r\, W$, where k is exclusive acquisitions per commit. A pre-registered intervention changing k supported this account, and the resulting read gain is paid for in writes (Section VI).
- **The writer's critical section limits the fix.** The cost of holding a read lock depends on how long it is held and on whether the writer is at capacity: below capacity, write throughput stays within the declared margin while median write response rises through two measured lock waits, and exploratory runs at a high write rate show reads following the time left outside the writer's hold (Sections V and VII).
- **Replacement designs are not interchangeable at the extremes.** Lease and Snapshot match on throughput within a declared margin across publication and compaction, and separate at high table counts under read-only load and in commit tails under compaction (Section VII).
- **A methodology lesson.** Back-to-back trials carried an fsync stall into the next trial, distorting tails while rechecked throughput classes held (Section VIII).

The six campaigns test pre-registered predictions; the original read crossover, admission and cycle accounts, tighter-margin sensitivity, compaction-tail and table-count interpretations, paced medians and backlog, carry-over diagnosis, fsync checks, and hold-share account are exploratory, with the median trend subsequently tested on fresh data.

II. Background

A. The lifetime problem and production designs

Stable references let a reader outlive a short coordination critical section. LevelDB takes references on memtables and the current version under its mutex, releases it while reading, and later releases the references; its writer also releases the mutex around log append and synchronization and batches queued writers [3]. RocksDB readers reference a SuperVersion, and Pebble readers reference a read state under a short `RLock` [4], [5]. These designs motivate the lifetime question; our implementations retain their engine's validation, file format, and publication path.

B. Reader-writer admission

Admission policy determines who proceeds after an exclusive hold. Reader-writer locks permit concurrent readers while excluding writers [12]; Go's `RWMutex` blocks new readers once a writer waits and releases queued readers on

TABLE I
LIFETIME DESIGNS WITH THE DEFAULT COMMIT PATH.

Property	CoarseLock	Lease	Snapshot
Memory-tier search under `Mu.RLock`	yes	yes	yes
Reader SSTable I/O under `Mu.RLock`	yes	no	no
Disk-reader lifetime	lock	table leases	set pin
Per-read lifetime synchronization	`Mu`	atomic leases	`snapshotMu`
New set descriptor on publication	no	no	yes
Conflict check and WAL sync under `Mu.Lock`	yes	yes	yes
Exclusive acquisitions per successful commit	2	2	2

writer unlock [18]. A continually waiting writer can therefore alternate with admitted readers. Phase-fair synchronization frames this ordering [14], without implying that Go satisfies its formal properties.

Earlier measurements used a defective reader and supply no pooled evidence for this study; Appendix C records that history and the withdrawn claims.

III. THE ENGINE AND THREE DESIGNS

A. Storage and point reads

All designs share the storage and corrected point-read paths. A skip-list memtable [11] orders versioned internal keys and flushes frozen contents into SSTables. Data blocks use a 4 KiB threshold, a flat sparse index, and a footer locating the index. The point reader checks cached user-key ranges, binary-searches the index, and issues one `ReadAt` per visited block, stopping after finding a visible version or passing the key. Versions can straddle blocks. Open-time range-metadata reads bypass the delay, and there is no application block cache.

B. Three ways to keep readers alive

CoarseLock protects file lifetime by retaining the manager read lock through lookup and I/O. All designs first search active and frozen memory under `Mu.RLock`; CoarseLock keeps it until the SSTable search ends, excluding publication and retirement throughout.

Lease protects individual file readers after releasing the manager lock. Under the read lock it copies table-reader pointers into reusable scratch storage and takes an atomic lease on every active table. It searches outside `Mu` and releases the leases afterwards. Shutdown closes admission and drains active and retired leases; timeout preserves descriptors that remain leased. Each read leases all active tables even when only one range can match.

Snapshot pins one immutable reader-set descriptor under the manager read lock and `snapshotMu`. It searches outside both locks and reacquires `snapshotMu` to release the pin. Publication allocates and installs a new set under the mutexes and retires the old set; once it drains, eligible closure and unlinking run outside both mutexes. Table I summarizes the designs.

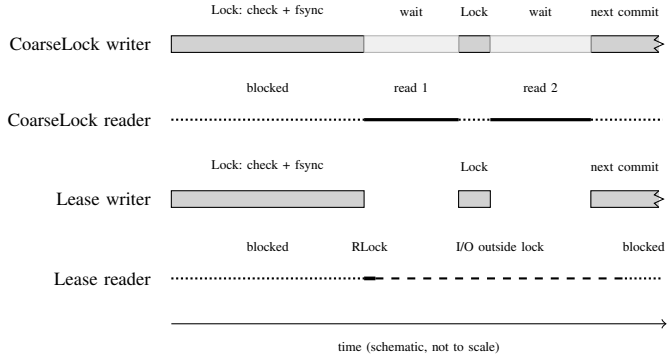


Fig. 1. One commit cycle under CoarseLock and Lease (schematic, not a measured trace). Under CoarseLock the writer waits for each read and each reader completes one read per exclusive acquisition. Under Lease the reader's I/O is outside the lock and can outlast the short second acquisition. CoarseLock: approximately two reads per reader per commit; Lease: fewer.

C. The commit path

The default commit path gives every design two exclusive acquisitions per successful commit, setting $k = 2$. `Transaction.Commit` first takes `Mu.Lock`, validates conflicts through `latestValueLocked`, appends and synchronizes the WAL, updates memory, and unlocks. It then calls `checkAndTriggerFlush`, which reacquires the exclusive lock even when no flush is needed. That second acquisition is unnecessary when no flush occurs; removing it below threshold changes k, as tested in Section VI-B. All designs share the default path. The first five campaigns time only the first acquisition's wait; the settled campaign times both.

Writer validation and fsync remain inside the exclusive hold after either reader-side fix. Conflict validation can read delayed SSTable blocks in every design. The delay therefore reaches the writer as well as readers, while the two acquisitions provide separate opportunities to admit queued readers (Fig. 1).

IV. METHOD

The campaigns fix the host and measurement conventions while varying read cost and write load. The host is an AMD Ryzen 7 7445HS with six cores, twelve logical CPUs, 15.2 GiB RAM, and a Samsung MZVL8512HFLU NVMe SSD, running Windows 11 Home build 26200 on NTFS with `GOMAXPROCS=12`. Timing uses QueryPerformanceCounter. Except for table-count variants, fixtures have 10,000 keys in four disjoint tables, 128-byte values, and uniform lookups with target hit ratio 0.95. Trials use 2 s reader-only warmup, a 10 s measurement window, and 200 ms cooldown.

The workloads separate writer capacity, publication, and lifetime work. Mixed uses a free-running writer and a 1 MiB memtable; Publication releases twenty pre-frozen memtables every 0.5 s; Churn uses a 16 KiB memtable and frequent flushing; Paced fixes intended write arrivals at fractions of earlier Lease capacity with closed or open readers. Compaction variants merge all active tables on a fixed timer, and table-count variants use disjoint ranges. Immediately before each point-read `ReadAt`, including writer validation, delay spins

with CPU yielding below 1 ms or sleeps then spins at 2 ms. The legacy scan is excluded from the ordered dose test. Appendix B gives full cells and coverage.

Pairing holds seeds and operation-index streams fixed within each cell-round unit, while randomized interleaving limits order effects. Faster designs may complete longer stream prefixes. Most campaigns use ten matched triplets per cell; the intervention uses five four-run units pairing two designs and two commit paths. Throughput counts in-window completions divided by the measured window, except the intervention's fixed 10 s denominator; latency histograms retain all in-window completions, while synchronization and flush counters include drain.

Service and response times distinguish execution from queueing. Service runs from issue to completion; response runs from intended arrival to completion. Below 0.95 completed/offered writes a trial is flagged saturated. Unissued ticks have no latency, so completion tails under overload are censored from below; a ratio with a censored component need not bound the true ratio. We use r for readers, d for delay, W, R for throughput, $S = W_{\mathrm{CL}}/W_{\mathrm{L}}$, $q = R/(rW)$, $T = 1/W$, and w for mean first-acquisition wait.

Equivalence uses paired log ratios and the declared margin $[0.90, 1/0.90]$, described as $\pm 10\%$ on the multiplicative scale. Holm-adjusted TOST establishes equivalence; a CI wholly beyond the margin is not equivalent, and remaining results are inconclusive. Trend tests use exact Page tests; Appendix A defines families, CIs, and sensitivity checks.

Six protocols were frozen before campaign trials, separating predictions from exploratory interpretation (Table II) [28]. The first five campaigns ran back to back without a deliberate pause, with a paced-load median gap of 0.04 s; the settled campaign added a pause, fresh seeds, and sentinel comparisons. All 2,349 retained trials succeeded on the first attempt, with no retries or dropped paired units; Appendix C records pre-freeze exposure and execution history.

V. How much does holding the lock cost?

A. A saturated writer pays for the read hold

CoarseLock's write penalty increased with measured read-lock hold (Fig. 2, Table III). At about 34 μs and four readers it completed 7.4% fewer writes than Lease, ratio 0.926 with 95% CI $[0.881, 0.974]$; around 163–173 μs losses were 24–29%, and around 1.9 ms they were 64–69%. H1 supported a decreasing ratio across doses at both reader counts, with Page's $L = 294$, exact one-sided $p = 7.86 \times 10^{-9}$, and ten paired blocks each. The point estimates were not strictly monotonic: at sixteen readers S rose from 0.937 to 0.944 between zero and 20 μs, with mean ranks 3.4 and 3.6. Below about 40 μs the largest point loss, 7.4%, is not a confidence bound: the 20 μs intervals were $[0.881, 0.974]$ and $[0.887, 1.004]$, allowing larger losses and, for sixteen readers, no loss. The legacy scan lost above 99% of writes at mean holds of about 125 and 430 ms.

Lease and Snapshot also slowed as delay increased in the exploratory comparison. Conflict validation reads SSTable blocks under the exclusive lock in every design, so Lease's mean commit cycle at four readers rose from 1.07 ms at zero delay to 2.79 ms at 2 ms. A shorter reader hold leaves that exclusive work in place.

B. Below capacity, the cost appears in medians

Below capacity, write throughput matched Lease while median write response rose with delay. H-P2 established equivalence in 6/6 paced cells. The earlier exploratory, descriptive p50 ratios were 1.093, 1.484, 2.681 with closed readers and 1.007, 1.336, 2.453 with open readers across zero, 200 μs, and 2 ms; the settled campaign then tested the trend on fresh data. H-M1 held in both modes with maximum Page's $L = 140$, exact $p = 1.65 \times 10^{-8}$, Holm-adjusted $p = 3.31 \times 10^{-8}$, and every round in the predicted order. Its response p50 ratios were 1.080, 1.431, 2.663 and 1.015, 1.332, 2.414, respectively. H-M2 held in all four higher-dose tests (Table IV). At zero delay there was no prediction; ratios 1.080 [1.067, 1.092] and 1.015 [0.998, 1.032] had 90% CIs inside the margin.

Both lock waits behind the median penalty were measured directly, supporting H-M3. At 2 ms with closed readers, CoarseLock's first wait was 1.952 [1.911, 1.993] ms and its flush check 2.105 [2.099, 2.112] ms, with 95% CIs, against 0.0003 and 0.027 ms for Lease. The exploratory phase account assigns about 82–93% of the excess total commit p50 at 200 μs and 2 ms to the two waits; medians do not add. Figure 3 separates achieved throughput from service medians; Appendix B retains both reader modes and full paced results.

C. Past capacity, the queue is unstable

H-P3 failed at 200 μs because its offered rate placed capacity next to the equivalence boundary. It met only 4/6 predictions: zero-delay cells were equivalent and 2 ms cells not equivalent, with ratios 0.390 and 0.405, but 200 μs was inconclusive in both modes, 0.896 [0.866, 0.927] and 0.920 [0.898, 0.942] with 90% CIs. CoarseLock fell below the completion flag in 19 of twenty trials there. The flaw was visible before freezing: earlier capacity divided by offered rate was $545.8/619 = 0.882$, already next to the 0.90 margin. H-W repaired the design at 700 writes/s: ratios 0.792 [0.775, 0.808] and 0.840 [0.818, 0.863], with 90% CIs, were both not equivalent.

Overload produced a window-dependent backlog in the exploratory paced comparison. At 2 ms and $f = 0.8$, mean completion p99 was about 6.05 s, close to $10(1 - 111.0/286) = 6.12$ s. This is an unstable queue, so its completion-only backlog percentile is not steady-state latency.

D. The pre-registered p99 trend failed

H-P1 did not support the predicted rise in the write response p99 ratio below capacity. At $f = 0.2$, closed readers gave Page's $L = 123$, $p = 0.291$, adjusted $p = 0.291$; open readers gave $L = 128$, $p = 0.047$, adjusted $p = 0.093$. At $f = 0.8$, the respective L values were 139 and 138, with adjusted $p = 1.4 \times 10^{-6}$ and 1.0×10^{-5}; every round had a censored component.

TABLE II
PRE-REGISTERED RESULTS. H3 HAD NO DIRECTIONAL PREDICTION; ITS OUTCOME IS CI REPORTING. R-CHECK IS THE SENTINEL RULE, DISTINCT FROM COMPACTION H-C.

Campaign	Prediction in plain words	Outcome	Section
Initial H1	Write ratio falls as delay rises	Supported	V
Initial H2	Lease/Snapshot throughput and publication waits match the margin	Partly supported	VII
Initial H3	Churn: no directional prediction; report paired CIs	No directional prediction	VII
Intervention P1	CoarseLock reads per reader per commit follow acquisition count	Supported	VI
Intervention P2	Removing an acquisition lowers the relative read ratio	Supported	VI
Intervention P3	Cycle errors fit the band; higher-dose writes improve	Supported	VI
Compaction H-C	Lease/Snapshot throughput matches with compaction on	Supported	VII
Table count H-T1	Lease/Snapshot read ratio declines with table count	Partly supported	VII
Table count H-T2	Equivalence at small counts; read-only separation at the largest	Supported	VII
Paced H-P1	CoarseLock/Lease write response p99 ratio rises with delay	Failed	V
Paced H-P2	Below-capacity write throughput matches the margin	Supported	V
Paced H-P3	Higher-load writes match at zero delay and separate at higher doses	Partly supported; failed at 200 μs	V
Paced H-P4	Lease/Snapshot write and closed-reader throughput match	Partly supported	VII
Settled H-M1	Below-capacity write response median ratio rises with delay	Supported	V
Settled H-M2	Median ratio lies above the margin at higher doses	Supported	V
Settled H-M3	Both CoarseLock waits fit the delay band; Lease waits are short	Supported	V
Settled H-W	Higher offered rate separates write throughput	Supported	V
Settled H-R	Previously inconclusive Lease/Snapshot reads and writes match	Failed	VII
Settled H-S	Below-capacity median largest fsync stays below the ceiling	Supported	VIII
Settled R-check	Recheck prior metrics and classes; rerun if the rule triggers	No rerun triggered; five metrics inconclusive	VIII

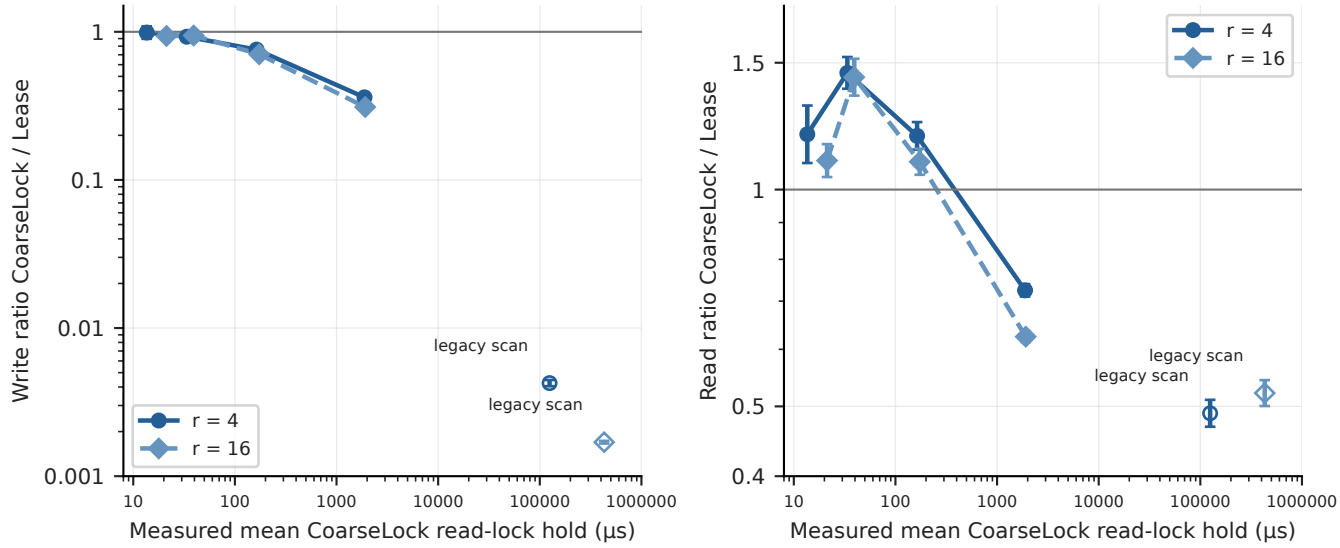


Fig. 2. Mixed workload: paired CoarseLock/Lease ratios against CoarseLock's arithmetic mean trial read-lock hold. Bars are 95% paired log-t CIs. Lines connect the four injected doses; hollow legacy scan points are unconnected. Measured hold is descriptive, not a separately randomized treatment.

Those tests detect increasing completion-only tails associated with backlog, not increasing true offered-work p99s.

A few stalls made p99 fragile in the exploratory diagnosis. Cell/design median maximum write service lay in 53–74 ms, with individual maxima 32–103 ms across doses and designs. At zero delay and $f = 0.2$, p99 concerns roughly the 18th-worst of about 1830 writes; a stall spans about 10–13 arrival ticks at 183 writes/s. Closed-reader zero-delay per-round p99 ratios ranged from 0.03–105.76. An engine-free append/fsync probe had maxima 12.8–36.5 ms over 6 ten-second runs, showing host stalls without accounting for the larger engine band. Section VIII diagnoses that band and reports the tails after pausing.

VI. ADMISSION COUPLING

A. *Reads follow exclusive acquisitions*

The exploratory read crossover appeared while CoarseLock wrote less. With a saturated writer it read more than Lease through the 200 μs dose and less at 2 ms: four-reader ratios ranged from 1.19 to 1.45 before falling to 0.725, and sixteen-reader ratios from 1.09 to 1.43 before falling to 0.625 (Fig. 2). This crossover was unpredicted.

Admission coupling accounts for the crossover under CoarseLock with a continually waiting writer. If each exclusive acquisition admits approximately one read per reader, then

$$R_{\mathrm{CL}} \simeq k\, r\, W_{\mathrm{CL}}, \qquad k = 2 \text{ on the default path.} \tag{1}$$

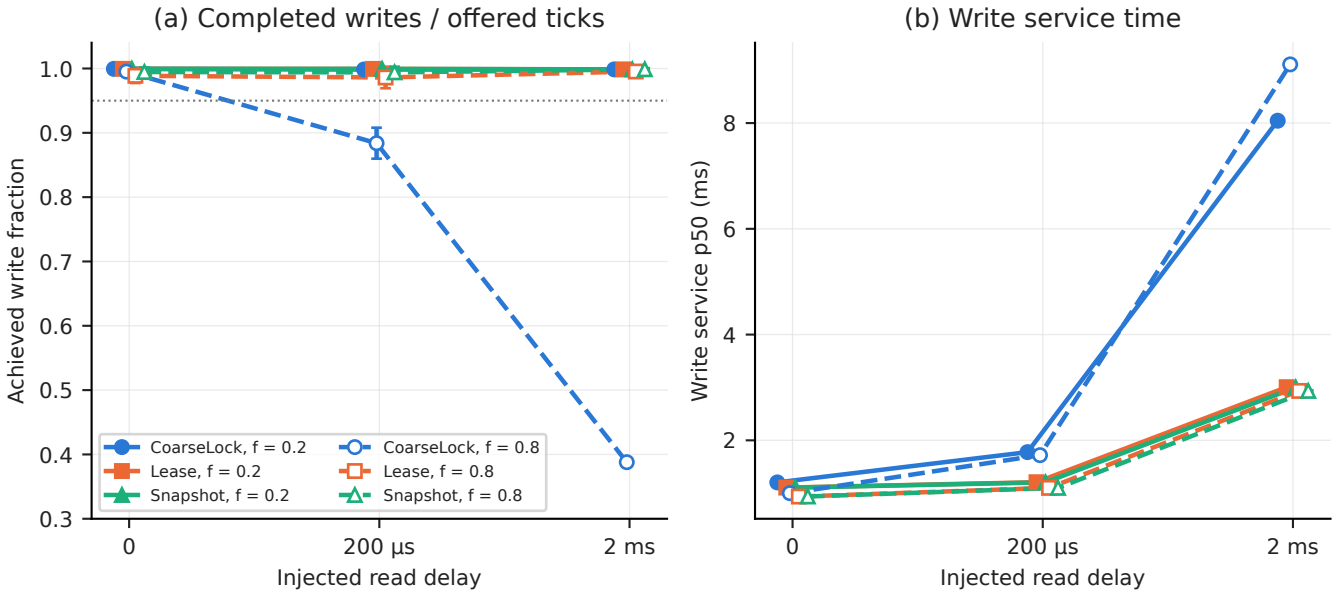


Fig. 3. Paced workload, closed-reader cells only. Points are arithmetic means of trial summaries, with frozen 95% t CIs. Solid filled markers show $f = 0.2$ and dashed hollow markers $f = 0.8$. Panel (a) marks the 0.95 completion flag; panel (b) shows write service p50, excluding scheduling delay. Open-reader cells showed the same broad pattern and are reported in the tables.

TABLE III
MIXED WORKLOAD: MEASURED COARSELOCK HOLD AND PERCENTAGE CHANGE RELATIVE TO LEASE, USING PAIRED GEOMETRIC RATIOS. THESE ARE POINT ESTIMATES; APPENDIX B GIVES CIS.

r	Delay	Hold (μs)	Writes	Reads
4	0	13.6	-1.2%	+19.3%
4	20 μs	33.6	-7.4%	+45.2%
4	200 μs	163	-24.2%	+18.7%
4	2 ms	1,885	-63.9%	-27.5%
4	legacy scan	124,853	-99.6%	-51.1%
16	0	21.2	-6.3%	+9.7%
16	20 μs	39.3	-5.6%	+43.2%
16	200 μs	173	-29.5%	+9.3%
16	2 ms	1,915	-68.9%	-37.5%
16	legacy scan	430,130	-99.8%	-47.8%

Individual trials in the eleven non-legacy Mixed and Churn cells had $q = 2.002$–2.015, despite changing delay and reader count. Legacy q ranged from 1.98 to 2.29 with only about 26 commits per trial at four readers and six at sixteen.

Lease and Snapshot can miss the second admission while doing I/O outside the manager lock. Their cell mean q fell from about 1.7 at zero delay to about 1.3 at 20 and 200 μs and about one at 2 ms (Table V). At four readers and zero delay, read p50 was 45 μs for CoarseLock and 808 μs for Lease, while p99 was 2.27 and 2.31 ms; inferred bimodality combines a fast read and one blocked behind a commit.

A descriptive cycle check is consistent with two reader waits per commit:

$$\widehat{T}_{\mathrm{CL}} = T_{\mathrm{L}} + 2w_{\mathrm{CL}}. \tag{2}$$

Arithmetic means of per-trial cycles gave prediction errors from -13.8% to $+10.6\%$ across ten Mixed cells. The second wait was assumed equal to the measured first wait; the direct measurements in Section V are below capacity.

B. Changing acquisition count tests the account

The pre-registered intervention supported coupling by changing k below the flush threshold. An experiment-only flag, false by default, checks the threshold inside the first exclusive hold and skips the second acquisition when no flush is needed. At or above threshold the default path reacquires and rechecks before rotation and flush. Validation and WAL work remain shared. Five paired four-run units per cell compare CoarseLock and Lease on both paths, with the default rerun in the same session.

All P1–P3 criteria held (Fig. 4). P1 required mean CoarseLock q near acquisition count: individual trial ranges were 2.002–2.013 for $k = 2$ and 1.001–1.006 for $k = 1$. P2's six applicable ratio-of-read-ratios CIs lay below one; at four readers and 20 μs the ratio was 0.659, 95% CI $[0.595, 0.730]$. P3's cycle errors were -3.7% to $+7.5\%$ with two acquisitions and -2.0% to $+8.4\%$ with one, within the declared band; all four higher-dose write-improvement CIs exceeded one. Appendix A gives the exact predictions and Appendix B the full estimates.

Removing an acquisition traded write gains for reader throughput and tails in the exploratory comparisons. Absolute one-acquisition/two-acquisition read ratios were 0.507–0.763 for CoarseLock and 0.608–0.850 for Lease. Through 200 μs both then had $q \simeq 1$, making the read ratio close to S; CoarseLock's read-ratio CIs included one at the two lowest

TABLE IV

SETTLED CAMPAIGN, TEN ROUNDS PER CELL. THE WRITE RESPONSE P50 RATIO IS THE PAIRED GEOMETRIC COARSELOCK/LEASE MEAN WITH ITS 90% CI; PHASES ARE ARITHMETIC MEANS OF PER-TRIAL P50 IN MS; THE LAST COLUMN IS THE CELL MEDIAN OF THE TRIAL LARGEST FSYNC IN MS. AT 700 WRITES/S COARSELOCK IS PAST CAPACITY, SO ITS RESPONSE P50 IS A COMPLETION-ONLY BACKLOG FIGURE AND THOSE RATIOS ARE CENSORED, NOT TRUE LATENCY RATIOS.

			CoarseLock/Lease	CoarseLock		Lease		Largest
Delay	Writes/s	Readers	Response p50 [90% CI]	Lock wait	Flush check	Lock wait	Flush check	fsync
0	183	closed	1.080 [1.067, 1.092]	0.075	0.074	0.001	0.047	21.8
0	183	open	1.015 [0.998, 1.032]	0.000	0.066	0.000	0.062	11.4
200 μs	155	closed	1.431 [1.398, 1.464]	0.229	0.296	0.000	0.033	9.8
200 μs	155	open	1.332 [1.325, 1.340]	0.137	0.264	0.000	0.028	8.9
2 ms	72	closed	2.663 [2.640, 2.687]	1.952	2.105	0.000	0.027	6.9
2 ms	72	open	2.414 [2.372, 2.456]	1.846	2.088	0.000	0.018	7.2
200 μs	700	closed	599.389 [195.644, 1836.337]	0.268	0.281	0.001	0.032	31.5
200 μs	700	open	206.265 [54.672, 778.187]	0.226	0.262	0.000	0.031	31.5
0	732	closed	1.114 [1.009, 1.231]	0.060	0.065	0.003	0.042	31.5

TABLE V

EXPLORATORY ADMISSION AND CYCLE CHECK: ARITHMETIC MEANS OVER TEN TRIALS PER CELL. CYCLES ARE MS; ERROR IS $100(\widehat{T}/T - 1)$. CHURN REPORTS ADMISSION RATIOS ONLY.

Workload	r	Delay	q_{CL}	q_{L}	q_{S}	T_{CL}	$\widehat{T}_{\mathrm{CL}}$	Error (%)
Mixed	4	0	2.009	1.664	1.657	1.077	1.130	4.9
Mixed	4	20 μs	2.003	1.278	1.277	1.140	1.175	3.1
Mixed	4	200 μs	2.004	1.280	1.280	1.627	1.738	6.8
Mixed	4	2 ms	2.003	0.998	1.001	7.712	7.432	-3.6
Mixed	4	legacy scan	2.032	0.018	0.018	384.3	363.6	-5.4
Mixed	16	0	2.010	1.717	1.718	1.169	1.249	6.8
Mixed	16	20 μs	2.008	1.323	1.318	1.206	1.334	10.6
Mixed	16	200 μs	2.006	1.295	1.294	1.835	1.978	7.8
Mixed	16	2 ms	2.003	0.995	0.996	9.008	8.726	-3.1
Mixed	16	legacy scan	2.209	0.007	0.007	1,666.7	1,437.3	-13.8
Churn	1	0	2.009	1.080	1.081	–	–	–
Churn	4	0	2.009	1.100	1.099	–	–	–
Churn	16	0	2.013	1.104	1.101	–	–	–

doses and fell below one at 200 μs. These crossings were not separately predicted. Lease write ratios were 0.970–1.044, with all CIs including one, and its one-acquisition q fell to 0.857–0.858 at 2 ms. Paired trial-p99 ratios were 1.330–2.049 for CoarseLock and 1.578–2.125 for Lease, descriptive raw-percentile summaries without CIs. Thus the default path's second acquisition admits readers as well as delaying writes.

C. Replication and the equal-write-rate bound

The exploratory paced counts replicated the coupling out of sample under closed readers. At $f = 0.8$, CoarseLock completed 547.2 writes/s at 200 μs and 111.0 at 2 ms, against earlier means 545.8 and 111.0. Reads were 17,560 versus $2rW = 17{,}511$, and 3,557 versus 3,552. Open readers have paced arrivals, so Equation 1 does not apply to their rates.

The read gain did not appear at equal write rates in the descriptive comparison. With closed readers, $f = 0.2$, no flushes, and matched achieved writes, CoarseLock/Lease read ratios were 0.967 [0.912, 1.025], 0.921 [0.889, 0.954], 0.720 [0.711, 0.730] across doses, with 95% CIs; at $f = 0.8$ they were 0.776 [0.666, 0.904], 0.550 [0.491, 0.616], 0.642 [0.632, 0.653]. The saturated-writer gain is therefore paid for in writes.

The exploratory compaction comparison exposed an unpredicted limit to the earlier read-gain account. At 2 ms the CoarseLock/Lease read ratios with compaction were 0.912 [0.892, 0.933] and 0.820 [0.806, 0.835] at four and sixteen readers, with 95% CIs, versus 1.154 and 1.081 without compaction. A post hoc, untested contributor is table count: off-condition CoarseLock flushed less and ended with cell means 8.0 tables versus Lease's 12.0. Admission and table-count contributions were not separated.

VII. WHEN DO LEASE AND SNAPSHOT DIFFER?

A. Throughput matches across publication

Lease and Snapshot met H2's throughput equivalence criterion in every Mixed and Publication cell. All ten write and fourteen read tests passed after Holm correction (Fig. 5), and all corresponding bootstrap intervals lay inside the declared margin. Publication at zero delay still had read ratio 0.970, 90% CI $[0.962, 0.978]$, a detectable roughly 3% difference within the margin. The exploratory tighter-margin check found 6 of ten write and 11 of fourteen read CIs inside $[0.95, 1/0.95]$; all failures of containment occurred at $d \leq 20$ μs, without a new adjusted equivalence test.

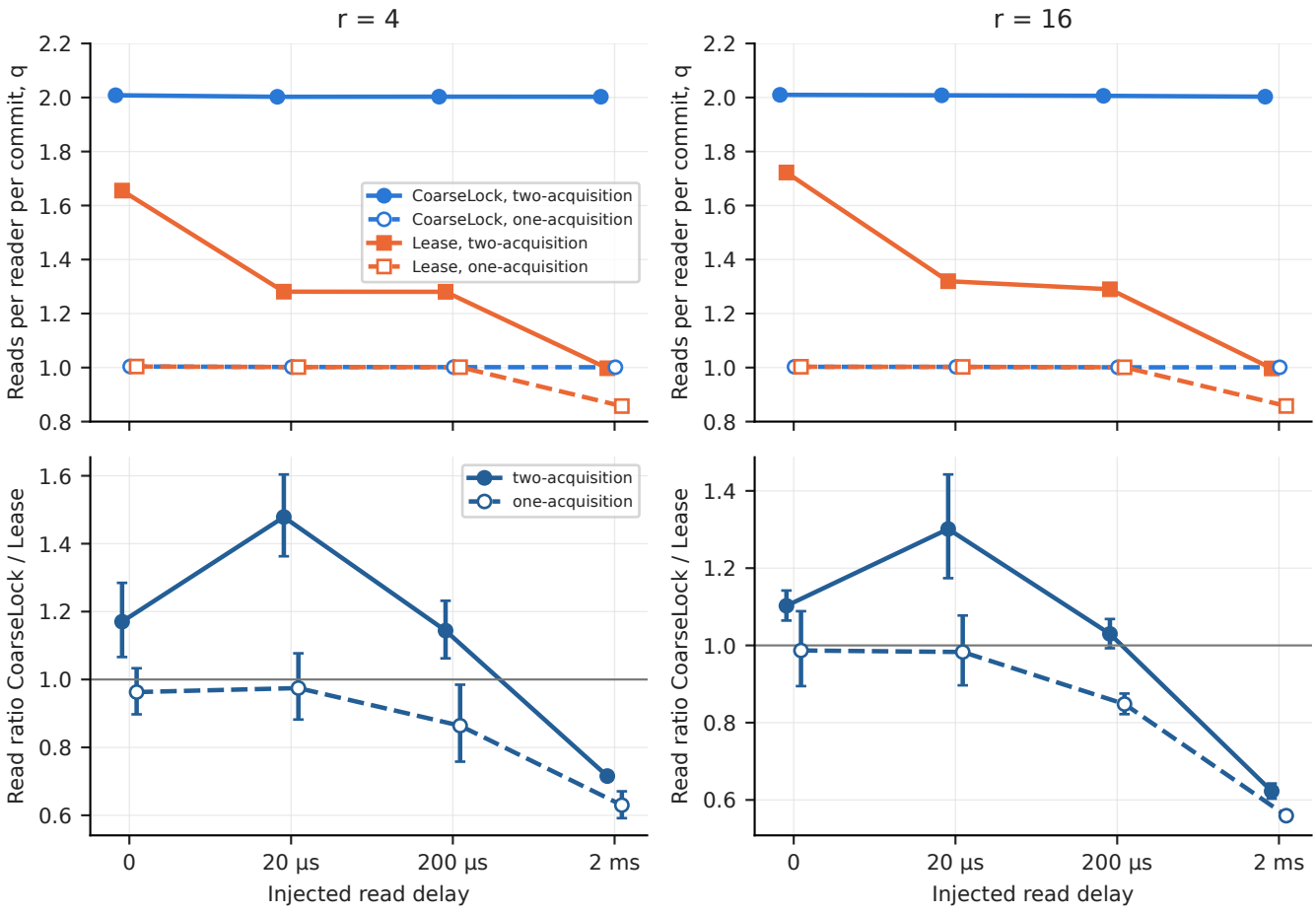


Fig. 4. Intervention, five paired four-run units per cell. Top: arithmetic mean reads per reader per commit. Bottom: paired geometric CoarseLock/Lease read ratios with 95% log-t CIs. Solid filled markers denote two-acquisition commits; dashed hollow markers denote one-acquisition commits below threshold.

Publication completion differed only under the legacy scan, while publication waits were not established as equivalent. All designs completed twenty scheduled publications at non-legacy doses; under the legacy scan CoarseLock completed median 10.5, range 9–11, while Lease and Snapshot completed twenty. Their cell geometric publication-wait p99s stayed below 42 μs, but the zero-delay Lease/Snapshot ratio was 3.11, 90% CI $[1.54, 6.30]$, not equivalent; the other three cells were inconclusive (Appendix B).

B. Churn corrects the lifecycle attribution

The Lease control does not support attributing Churn's read deficit to Snapshot's lifecycle, an exploratory earlier reading. CoarseLock/Snapshot read ratios were 1.66, 1.53, and 1.29 across reader counts, and Lease also showed the deficit despite allocating no reader-set descriptors. H3 had no directional prediction: all Lease/Snapshot 95% paired-t CIs included one, while bootstrap intervals at one and sixteen readers narrowly excluded it for both outcomes, including the one-reader read interval $[0.909, 0.997]$; these data do not establish that Lease avoids the deficit.

C. Compaction separates commit tails

Compaction preserved throughput equivalence and separated commit-wait tails. All twelve primary on tests met H-C after separate write and read Holm corrections, as did the twelve off reference tests. The closest write comparison was 0.949, 90% CI $[0.909, 0.991]$, at sixteen readers and 200 μs.

The exploratory tail comparison gave Lease commit-wait p99 ratios of 1.43–7.69 times Snapshot's, with every 95% paired CI excluding one (Fig. 6). Absolute arithmetic means were 21–246 μs for Lease, 12–177 μs for Snapshot, and 306–9221 μs for CoarseLock. CoarseLock and Lease open compaction inputs under the exclusive lock, while Snapshot opens them outside all locks; this asymmetry is not an established cause of the tail gap.

Turning compaction on raised write throughput by 1.17–2.66 and reads by 1.31–2.86 in the exploratory on/off comparison, while reducing overlapping tables from a maximum cell mean 70.0 to means 1.0–2.8; the comparison includes relief from read amplification.

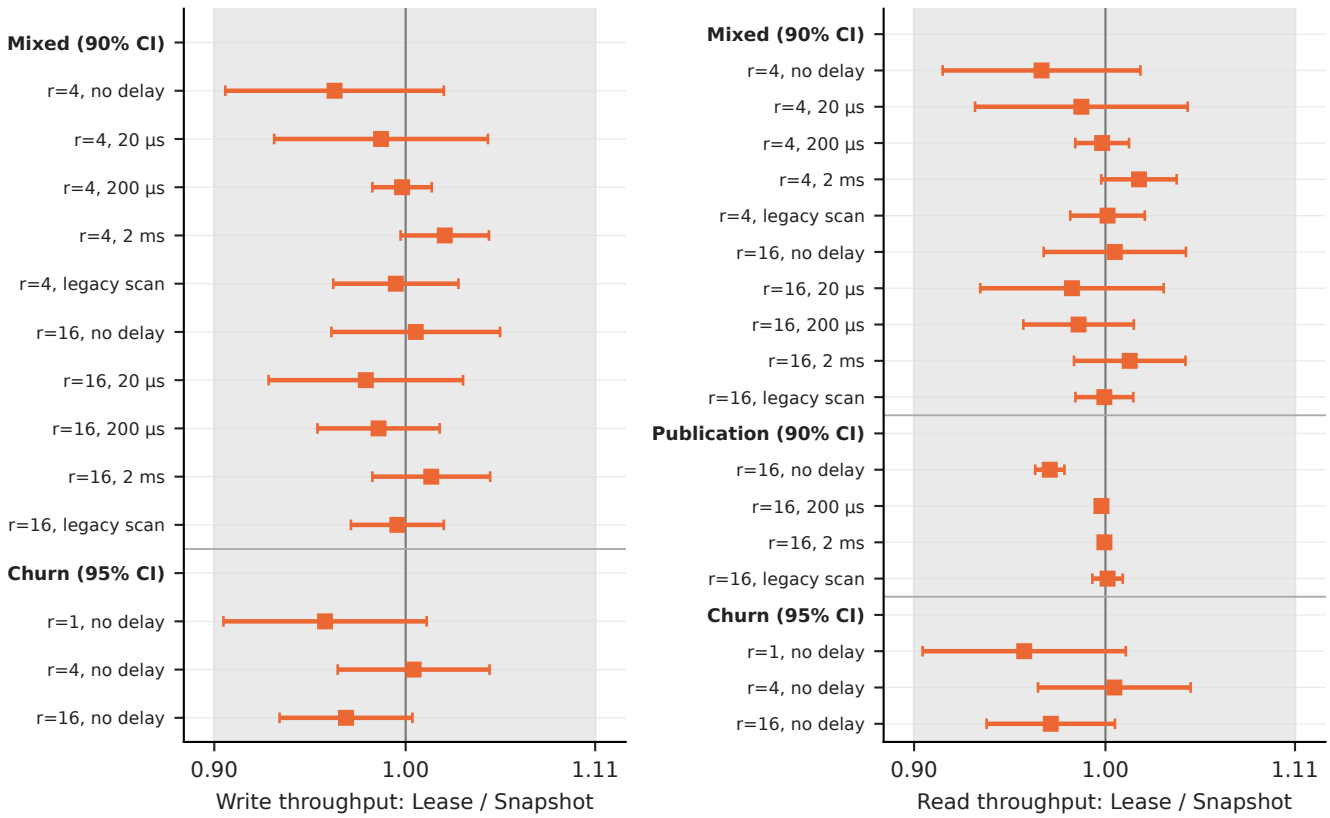


Fig. 5. Lease/Snapshot throughput, ten triplets per cell. Points are paired geometric ratios; bars are 90% paired log-t CIs for Mixed and Publication, and 95% for Churn. Shading shows the rounded $[0.90, 1.11]$ margin. Churn was not an equivalence test.

D. Table count exposes per-read work

Read-only load separated Lease and Snapshot as table count grew, while free-writer throughput remained equivalent. H-T1 detected declining read ratios in both read-only families, Page's $L = 295$ and 296 with Holm-adjusted $p = 5.88 \times 10^{-9}$ and 1.60×10^{-9}, but not with a writer. All H-T2 predictions held: equivalence at four tables and read-only non-equivalence at 256 for both reader counts. Across the four counts, read-only ratios were 0.975, 0.956, 0.918, 0.712 at four readers and 0.989, 0.962, 0.851, 0.663 at sixteen. At sixteen readers, 64 tables already gave 0.851, 90% CI $[0.837, 0.866]$; 256 gave 0.663, 90% CI $[0.660, 0.666]$, about 34% fewer reads. All sixteen applicable free-writer tests were equivalent, with ratios 0.943–1.045 (Fig. 7).

The exploratory critical-section evidence locates growing work in Lease's read path. Read-only mean holds grew from 326–3370 ns at four readers and 393–8512 ns at sixteen, while Snapshot stayed below 1 μs. Separate capture/release microbenchmarks cost Lease 27–974 ns at one goroutine and 58–1767 ns at sixteen, against Snapshot's 21 and 88–89 ns. Publication cost Lease 104–128 ns with no allocations and Snapshot 654–3200 ns with three, moving table-proportional work from reads to publication (Appendix B).

E. A high write rate leaves little time for readers

H-P4 and H-R left the same read comparison inconclusive twice. H-P4 established Lease/Snapshot write equivalence in 12/12 cells and closed-reader read equivalence in 5/6. At zero delay and $f = 0.8$, reads were 1.072 [0.910, 1.264], 90% CI, adjusted $p = 0.35$, with log-ratio SD 0.284. On fresh settled data H-R failed: writes were equivalent, 1.001 [0.994, 1.008], but reads remained inconclusive, 0.995 [0.846, 1.171], adjusted $p = 0.142$, SD 0.280.

The exploratory fsync on/off check reduced read spread at the same frozen load specification. Pooled log-read SD fell from 0.142 to 0.018, a ratio of 0.128 [0.086, 0.191] with a 90% F interval assuming normality; the read comparison moved from inconclusive to equivalent (Appendix B). Fsync off removes ordinary holds as well as bursts, implicating fsync without isolating bursts.

The exploratory hold-share account relates reads to time outside the writer's exclusive hold (Fig. 8). At 732 writes/s, median hold shares were 74.7% in the campaign, range 67.4–79.2, and 72.3% in the check, range 67.6–78.8, versus 4.3% with fsync off. Log reads tracked log free share with $r = 0.905$, slope 1.11 in the campaign and $r = 0.965$, slope 1.16 in the check; bursts occupied median 4.9–8.3% of the window. Adjusting the paired campaign ratio for free share tightened

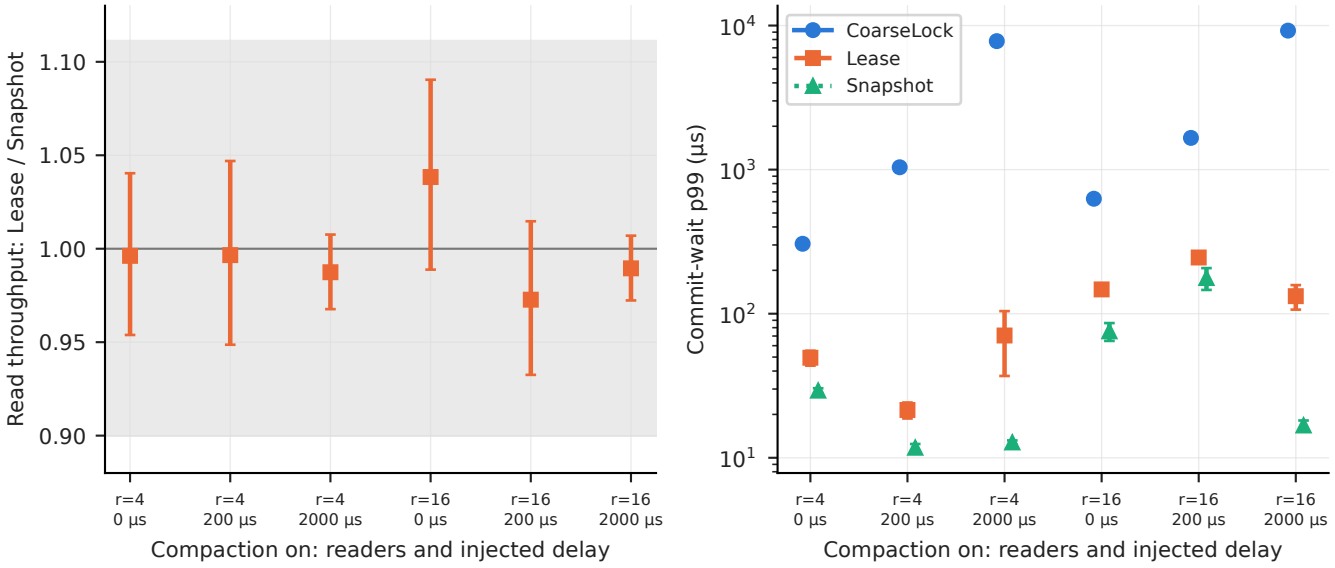


Fig. 6. Compaction-on cells, ten triplets each. Left: Lease/Snapshot read throughput with 90% paired log-t CIs and the equivalence band. Right: arithmetic mean trial commit-wait p99 with 95% t CIs, in μs on a log axis. Ratio intervals and absolute-duration intervals use different summaries.

0.995 [0.846, 1.171] to 1.003 [0.956, 1.052] and reduced log-ratio SD from 0.280 to 0.082. This adjusted CI is not a test of H-R, which stays failed; hold time is an outcome of the same trial and direction is not established.

VIII. Trial Carry-Over: A Methodology Lesson

A. Pauses removed the carry-over band

The exploratory diagnosis located the large write stall inside WAL fsync and linked it to the preceding trial. All 27 fsync-on diagnostic runs placed the largest commit's time there. In 9/9 same-specification pairs, the largest fsync was higher back to back: median 54.9 ms, range 45.7–73.0, versus 8.0 ms, range 5.7–17.0, after 30 s pauses, with nonoverlapping ranges. Deleting a trial-shaped directory alone did not reproduce the effect. Calibration pauses of at least 15 s removed the band above about 45 ms in 18/18 trials; the settled protocol froze 20 s.

H-S supported the paused protocol's below-capacity ceiling. Realized pauses were 20.003–20.030 s, with cell medians of the largest fsync at 6.9–31.5 ms. Cell medians of largest write service descriptively fell from 54–59 to 9.5–22 ms; one trial still reached 81.3 ms.

B. Rechecked classes held while tails moved

R-check triggered no rerun and contradicted no rechecked published class. Of 22 metrics, 17 were unchanged, 5 inconclusive, and 0 changed; none of 9 published classes was contradicted. The five inconclusive metrics were the Publication lock-wait p99 Lease/Snapshot ratio, the compaction commit-wait p99 Lease/Snapshot ratio, and three five-round intervention throughput ratios whose intervals extended beyond the margin; they do not rule out change. The compaction tail ratio moved from 1.43 to 2.40, as Snapshot's median trial p99 fell from 0.177 to 0.104 ms while Lease's remained near 0.245 and 0.232 ms.

The exploratory tail recheck showed substantial distortion at 2 ms and unusable zero-delay tails after pausing. CoarseLock cell-median response p99 fell from 25.95 to 11.17 ms with closed readers and 34.41 to 11.70 ms with open readers; geometric CoarseLock/Lease ratios fell from 4.04 to 1.85 and 4.98 to 1.64. H-P1 stands as failed. Zero-delay p99 log-ratio SD remained 3.16 and 2.05.

C. The bare-loop prediction failed

The exploratory burst check failed to reproduce the engine pattern from rate alone. Fsyncs near 31 ms became more common with engine write rate: trials with an fsync of at least 25 ms rose from 6/60 at 72 writes/s to 88/90 at 650–730 writes/s (Appendix B). The engine-free append/fsync prediction, at least 8 of 10 runs with such an fsync at 700 writes/s, failed: only 5 of 10 did; rate/burst Spearman correlation was 0.40, unadjusted $p = 0.010$. In 5 bare runs across rates the only slow fsync was the first, at 47.4–51.8 ms. The 4 runs with real bursts were all among the first ten of forty despite shuffled rates. Flushes, readers, data volume, and run order, with bare runs first, were not separated, and the cause remains unknown. Pause between storage trials, report the gap, and check results against a paused sentinel.

IX. Discussion

A reader-side lifetime fix helps when writers wait on lock-held lookups at the intended write load or those waits limit capacity. Admission coupling makes the read/write balance part of that decision, and the writer's own exclusive work bounds either replacement. Five steps follow:

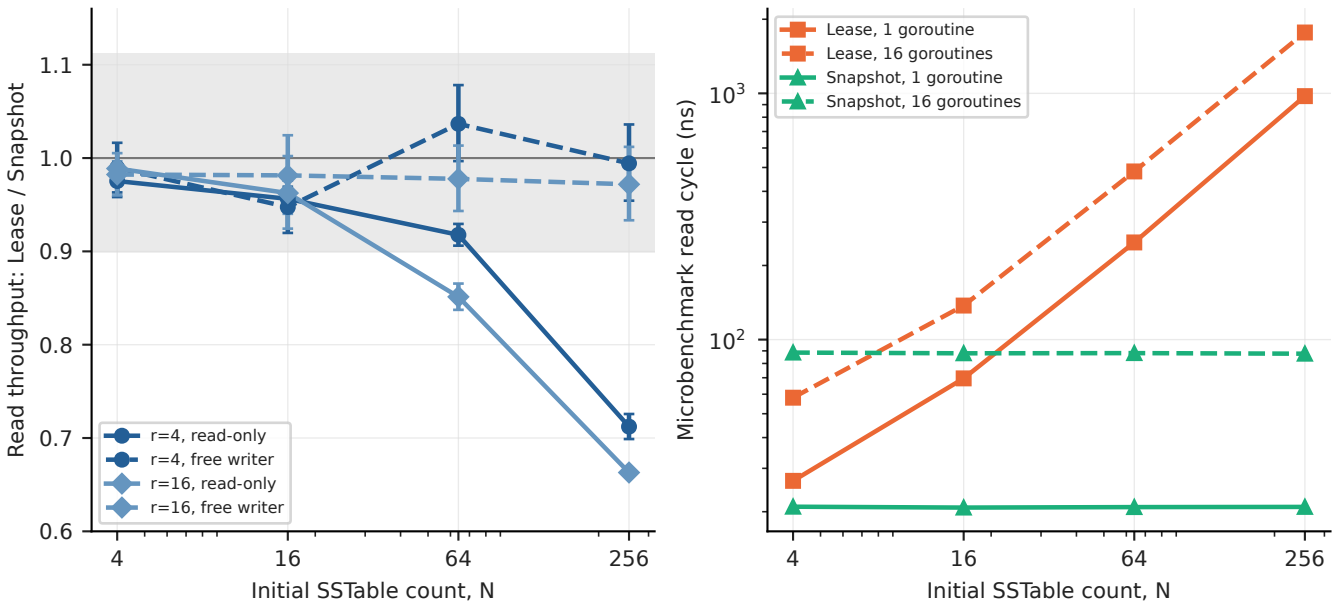


Fig. 7. Table-count campaign and separate lifecycle microbenchmark. Left: Lease/Snapshot read ratios with 90% paired log-t CIs; solid lines are read-only and dashed lines have one free writer. Right: arithmetic means of three benchmark counts, with no campaign CI. The benchmark isolates capture/release and excludes search and I/O.

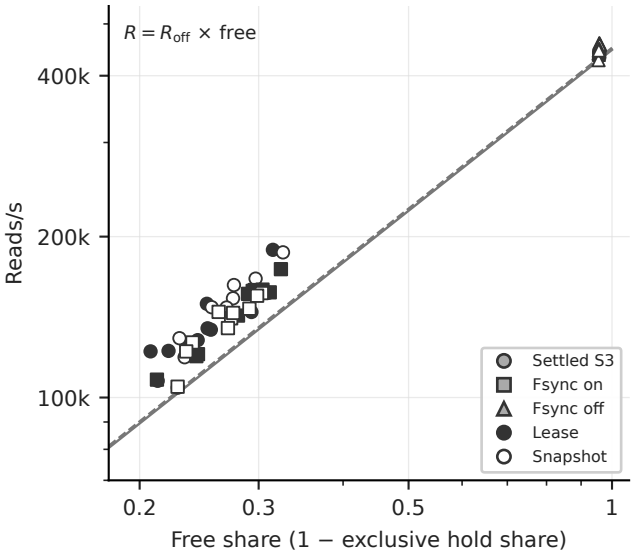


Fig. 8. Reads against the share of the window outside the writer's exclusive hold, zero delay, 732 writes/s, closed readers, log axes. Post hoc, exploratory. The line is fsync-off median capacity scaled by free share.

1) Measure lock-held lookup time and split writer time into validation, synchronization, and lock waits.
2) Compare capacity and response time at the intended offered write rate, including an equal-write-rate comparison.
3) Count exclusive acquisitions per commit and inspect reader admission before changing the commit path.
4) Compare replacement designs at the intended table count, publication rate, and reader load, including tails.
5) Measure the writer's hold share and verify trial spacing with a paused sentinel.

A. *What we did not test*

Further experiments must isolate overlapping-table search, independently queued writes, multiple writers, longer windows, burst and carry-over triggers, fsync outside the lock with group commit, other admission policies, real device reads on larger datasets, and other operating systems. Intervening on compaction input opening could test the proposed explanation of the tail gap.

X. Threats to Validity

A. *Construct*

Injected delay controls read cost rather than reproducing a device. The small dataset was likely page-cache resident, with no application block cache; every visited block incurs delay, and the treatment changes from yielding spin to mostly sleep. Sixteen readers run on twelve logical CPUs. Closed-loop readers can understate latency relative to open arrivals [33]; paced readers add open load, but unissued write ticks still have no latency observation, including some trials above the completion flag. The overloaded queue and its censored tails depend on the measurement window. Synchronization and flush counters include drain, unlike throughput; publication p99 is near a maximum over twenty events, and inferred read bimodality has no full reported histogram.

Compaction and table-count variants isolate limited workload features. The first two campaigns had no compaction;

the later merge-all timer differs from production level- or size-based policies, and on/off comparisons also change read amplification. Disjoint table ranges shrink under a fixed keyspace, while every design still does range comparisons. Synthetic lifecycle microbenchmarks exclude search, memory-tier lookup, and I/O, prime scratch storage, and use a Lease helper and spare publication slice capacity.

B. Internal

Host state remains a source of variation despite pairing and randomized order [30]. First-campaign drift was $D = 0.979$, $|\ln D| = 0.021$, below 0.10; CoarseLock and Snapshot end/start read ratios were about 1.09 with logs 0.087 and 0.090, each based on single anchors. Later campaigns had no anchors. Back-to-back running affected the first five campaigns, and settled sentinels cover only rechecked cells while confounding the pause with date, rebuilt binary, tracing, and seeds. Bursts near 31 ms were uncontrolled. Settled execution deviations included helper activity, an overlapping status check, and the running desktop application; Appendix C records them in full.

The shared writer path constrains both mechanisms and attribution. WAL fsync remains under the exclusive lock without group commit; the separate exploratory fsync-off diagnostic reduced commit p50 from 1.114 to 0.083 ms for Lease, 1.106 to 0.080 ms for Snapshot, and 1.229 to 0.167 ms for CoarseLock, with three runs each. Delayed writer validation was noticed after collection, correcting the first protocol's reader-only statement. The default second acquisition remains common to all designs; the cycle check assumes its wait at saturation, while settled timing measures it below capacity. The compaction input-open asymmetry was disclosed before that campaign, is not intrinsic to leasing, and was not experimentally isolated. Smoke exposure, the aborted first start, power-overlay history, and the race-detector context are in Appendix C.

C. External

One engine on one Windows host, at most one writer, injected delay, and fsync under the exclusive lock define the study's scope. Production engines may use group commit, block caches, different validation and publication, multiple writers, or different admission policies. These implementations do not reproduce LevelDB, RocksDB, or Pebble. The coupling equation describes this Go lock and workload rather than a general readers-writers law; the sole experimenter and analyst has no independent replication.

D. Statistical

The equivalence margin is the author's judgment rather than a production requirement. Holm adjustment applies within declared families, not across all outcomes. Churn has no equivalence prediction; CIs including one do not establish equality, and a missing writer-present trend does not establish absence of per-table cost. The intervention has five paired units, omits Snapshot, and has no multiplicity correction, including P2. Its rerun baseline reproduced admission but not identical ratios: sixteen-reader 20 μs S was 0.855 versus 0.944 earlier, and cohorts were not pooled. Tighter-margin containment is post hoc and unadjusted; bootstrap sensitivity does not remove paired-inference assumptions.

New-data predictions preserve their exploratory origins. The admission account followed the first campaign, and settled hypotheses arose from earlier outcomes. The fsync checks are unadjusted; post hoc hold-share adjustment cannot establish causal direction or repair H-R. Free-share capacity scaling undershot reads by mean 0.14–0.19 in log, and hold/burst correlations differed, 0.44 versus 0.94, so the account is not a calibrated model and bursts do not explain all variation. Compaction-tail and cache interpretations remain untested; within-host repetition does not replicate machines or implementations.

XI. Related Work

LSM concurrency work pursues scalability across more engine components than reader lifetime. The LSM-tree and subsequent survey frame the storage design [1], [2]; cLSM uses non-blocking structures with shared-exclusive coordination around merge pointer updates [7], FloDB scales the memory component [8], and RocksDB's development history describes broader production priorities [6]. SILK schedules internal I/O to prevent latency spikes [9], while performance-stability work studies write stalls [10]. Our fixed-timer on/off comparison includes read amplification and does not decompose background-work cost.

Reclamation and synchronization research explain different parts of the trade-off. Hazard pointers and comparisons with epoch and quiescent-state methods address lifetime protection without a globally optimal scheme [19], [20]. Scalable, phase-fair, NUMA-aware, passive, and biased reader-writer locks address admission and reader costs [13]–[17]. Our coupling result connects admission to completed storage reads and writes through exclusive acquisitions per logical commit. Group commit and Aether's early lock release and flush pipelining address writer work that the reader-side replacements retain [21], [22].

Evaluation literature motivates separating throughput, latency, and workload construction. Benchmarking guidance identifies measurement and host-state pitfalls [29]–[32]; open versus closed load changes queueing interpretation [33]. Equivalence testing and pre-registration separate a declared tolerance from prediction and observation [27], [28]. The carry-over diagnosis adds a concrete reason to check storage-trial spacing against a paused sentinel.

XII. Conclusion

A reader-side lifetime fix helps when shortening the read hold relieves writer waiting at the intended load; the writer's own critical section bounds the gain. Admission coupling ties CoarseLock reads to exclusive acquisitions and writes, so its saturated-writer read gain is paid for in writes and did not appear at equal write rates. Below capacity the cost appears in median response through both lock waits, while

exploratory high-rate runs relate reads to time outside the writer's hold. Lease and Snapshot match throughput within the declared margin until read-only table count separates them, and commit tails differ under compaction. Back-to-back storage trials carried an fsync stall into the next trial; pausing changed tails while rechecked throughput classes held. These results cover one engine on one Windows host, at most one writer, injected delay, and WAL fsync under the exclusive lock.

Acknowledgment

Anthropic's Claude and OpenAI's Codex were used to assist with engineering, analysis scripts and drafting of text. The author reviewed the results and is responsible for the content.

Artifact Availability

The repository is https://github.com/itsramhere/selfdbms, with campaign records on branch `research/x-readcost`. Evidence is indexed by `paper/CLAIM_EVIDENCE_MAP_X.md`; the reproducibility appendix is `paper/REPRODUCIBILITY_APPENDIX.md`, and paper names map to repository identifiers in `paper/REPOSITORY_IDENTIFIERS.md`. Appendix A records protocols and provenance, Appendix B the full results, and Appendix C execution history.

References

[1] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.

[2] C. Luo and M. J. Carey, "LSM-based storage techniques: a survey," *The VLDB Journal*, vol. 29, no. 1, pp. 393–418, 2020.

[3] S. Ghemawat and J. Dean, "LevelDB." [Online]. Available: https://github.com/google/leveldb

[4] Meta Platforms, "RocksDB." [Online]. Available: https://github.com/facebook/rocksdb

[5] Cockroach Labs, "Pebble." [Online]. Available: https://github.com/cockroachdb/pebble

[6] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, "RocksDB: Evolution of development priorities in a key-value store serving large-scale applications," *ACM Trans. Storage*, vol. 17, no. 4, Art. 26, 2021.

[7] G. Golan-Gueta, E. Bortnikov, E. Hillel, and I. Keidar, "Scaling concurrent log-structured data stores," in *Proc. EuroSys*, 2015, Art. 32.

[8] O. Balmau, R. Guerraoui, V. Trigonakis, and I. Zablotchi, "FloDB: Unlocking memory in persistent key-value stores," in *Proc. EuroSys*, 2017, pp. 80–94.

[9] O. Balmau, F. Dinu, W. Zwaenepoel, K. Gupta, R. Chandhiramoorthi, and D. Didona, "SILK: Preventing latency spikes in log-structured merge key-value stores," in *Proc. USENIX ATC*, 2019, pp. 753–766.

[10] C. Luo and M. J. Carey, "On performance stability in LSM-based storage systems," *Proc. VLDB Endow.*, vol. 13, no. 4, pp. 449–462, 2019.

[11] W. Pugh, "Skip lists: A probabilistic alternative to balanced trees," *Commun. ACM*, vol. 33, no. 6, pp. 668–676, 1990.

[12] P. J. Courtois, F. Heymans, and D. L. Parnas, "Concurrent control with 'readers' and 'writers'," *Commun. ACM*, vol. 14, no. 10, pp. 667–668, 1971.

[13] J. M. Mellor-Crummey and M. L. Scott, "Scalable reader-writer synchronization for shared-memory multiprocessors," in *Proc. PPoPP*, 1991, pp. 106–113.

[14] B. B. Brandenburg and J. H. Anderson, "Reader-writer synchronization for shared-memory multiprocessor real-time systems," in *Proc. ECRTS*, 2009, pp. 184–193.

[15] I. Calciu, D. Dice, Y. Lev, V. Luchangco, V. J. Marathe, and N. Shavit, "NUMA-aware reader-writer locks," in *Proc. PPoPP*, 2013, pp. 157–166.

[16] R. Liu, H. Zhang, and H. Chen, "Scalable read-mostly synchronization using passive reader-writer locks," in *Proc. USENIX ATC*, 2014, pp. 219–230.

[17] D. Dice and A. Kogan, "BRAVO: Biased locking for reader-writer locks," in *Proc. USENIX ATC*, 2019, pp. 315–328.

[18] The Go Authors, "sync package: RWMutex." [Online]. Available: https://pkg.go.dev/sync#RWMutex

[19] M. M. Michael, "Hazard pointers: Safe memory reclamation for lock-free objects," *IEEE Trans. Parallel Distrib. Syst.*, vol. 15, no. 6, pp. 491–504, 2004.

[20] T. E. Hart, P. E. McKenney, A. D. Brown, and J. Walpole, "Performance of memory reclamation for lockless synchronization," *J. Parallel Distrib. Comput.*, vol. 67, no. 12, pp. 1270–1285, 2007.

[21] D. J. DeWitt, R. H. Katz, F. Olken, L. D. Shapiro, M. R. Stonebraker, and D. A. Wood, "Implementation techniques for main memory database systems," in *Proc. ACM SIGMOD*, 1984, pp. 1–8.

[22] R. Johnson, I. Pandis, R. Stoica, M. Athanassoulis, and A. Ailamaki, "Aether: A scalable approach to logging," *Proc. VLDB Endow.*, vol. 3, no. 1–2, pp. 681–692, 2010.

[23] E. B. Page, "Ordered hypotheses for multiple treatments: A significance test for linear ranks," *J. Amer. Statist. Assoc.*, vol. 58, no. 301, pp. 216–230, 1963.

[24] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *J. Pharmacokinet. Biopharm.*, vol. 15, no. 6, pp. 657–680, 1987.

[25] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[26] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.

[27] D. Lakens, "Equivalence tests: A practical primer for t tests, correlations, and meta-analyses," *Soc. Psychol. Personal. Sci.*, vol. 8, no. 4, pp. 355–362, 2017.

[28] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Natl. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, 2018.

[29] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. OOPSLA*, 2007, pp. 57–76.

[30] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!," in *Proc. ASPLOS*, 2009, pp. 265–276.

[31] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems," in *Proc. SC*, 2015, pp. 1–12.

[32] A. Traeger, E. Zadok, N. Joukov, and C. P. Wright, "A nine year study of file system and storage benchmarking," *ACM Trans. Storage*, vol. 4, no. 2, Art. 5, 2008.

[33] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proc. NSDI*, 2006.

APPENDIX A
PROTOCOLS

A. Hypotheses, margins, and statistical methods

The first campaign's H1 tests a decreasing S across the four non-legacy doses using the one-sided exact Page test [23], blocked by triplet. H2 uses two one-sided tests (TOST) [24], [27] on paired log Lease/Snapshot ratios. It reports exponentiated 90% paired-t CIs and applies Holm correction [25] separately to ten write, fourteen read, and four publication lock-wait p99 tests. An adjusted $p < 0.05$ establishes equivalence; a CI wholly beyond a margin is classified not equivalent, and other outcomes are inconclusive. H3 reports two-sided 95% paired-t CIs under Churn without a directional prediction. Descriptive ratios use 95% CIs. Percentile bootstrap sensitivity [26] uses 10,000 resamples and fixed seeds; it does not replace the planned tests. Cells are not pooled, and no planned write-zero adjustment was needed.

The margin is fixed before data at $\pm\ln(1/0.90)$, or ratios $[0.90, 1/0.90]$, described here as $\pm 10\%$ on the multiplicative scale. It is the author's judgment of a difference small relative to the complexity difference between designs, chosen so ten pairs could resolve it. It is not derived from a production requirement. The pre-registration used a pilot standard deviation of paired log ratio near 0.06, giving an expected 90% CI half-width near 0.035 at ten pairs. Full CIs let readers apply a tighter margin.

The new campaigns preserve this method while defining their own families. The compaction primary hypothesis (H-C) predicts equivalence in all six on cells for each of writes and reads, with separate six-test Holm families; off cells form separate reference families. The table-count trend hypothesis (H-T1) uses four Page tests, one per reader-count and writer-count combination, with Holm across those four. Its equivalence families (H-T2) contain sixteen read and eight applicable write tests. H-T2 predicts equivalence at four tables for each applicable outcome and non-equivalence for reads at 256 tables without a writer. The intervention has five paired units per cell and 95% CIs without multiplicity correction, as specified before data.

The paced-load primary hypothesis (H-P1) predicts an increasing CoarseLock/Lease write response p99 ratio with delay, using four Page tests and Holm across the load-fraction and reader-mode combinations. H-P2 predicts below-capacity write-throughput equivalence in six cells; H-P3 predicts equivalence at zero delay and a CoarseLock ratio below the margin at the two higher doses under the higher offered fraction, with separate six-test Holm families. H-P4 predicts Lease/Snapshot throughput equivalence in twelve write and six closed-reader tests, with separate Holm families and no open-reader read-throughput test. Median latency and Lease/Snapshot tails have no prediction. Censored components remain in the primary completion-tail test and are disclosed rather than treated as true p99s.

The settled primary hypothesis (H-M1) predicts a rising CoarseLock/Lease write response p50 ratio with delay at $f = 0.2$, using two Page tests with Holm across reader modes. H-M2 predicts non-equivalence, with CoarseLock higher, at 200 μs and 2 ms in both reader modes. H-M3 predicts that CoarseLock's lock-wait and flush-check p50s each lie in $[1, 3]$ ms at 2 ms with closed readers, and Lease's below 0.5 ms, with 95% t CIs wholly inside each band. H-W predicts CoarseLock/Lease write non-equivalence at 700 writes/s. H-R predicts Lease/Snapshot read and write equivalence in the previously inconclusive cell. H-S checks that the median largest fsync stays below 40 ms in every $f = 0.2$ cell and design. The sentinel rule R-check compares 22 metrics with their originals using a 90% Welch interval on the change, with margin $\ln(1/0.90)$ for throughput and $\ln 1.5$ for tails, and classifies each as unchanged, changed, or inconclusive. A changed metric or a contradicted published class commits us to rerunning that campaign. These hypotheses were generated from earlier data and tested on fresh seeds.

The five-round design pairs both conditions in CoarseLock and Lease within each cell. P1 predicts CoarseLock's arithmetic mean q in $[1.90, 2.10]$ under two acquisitions and $[0.90, 1.10]$ under one, in all eight cells. P2 predicts that removing the acquisition lowers CoarseLock's read ratio relative to Lease at delays up to 200 μs: the 95% paired-t CI of the ratio of read ratios must lie wholly below one in six cells. These are unadjusted five-pair comparisons, of descriptive strength as specified in the protocol. P3 predicts cycle error within $\pm 15\%$ for both conditions, using two waits or one as appropriate, and a CoarseLock write improvement with its 95% CI above one at 200 μs and 2 ms.

R-check is the paper name for the settled sentinel rule stored under frozen JSON key `H_C` in `research/experiments/phase_i7/X_READCOST/analysis_settled/x_settled_results.json`. The frozen `research/experiments/phase_i7/X_SETTLED_PREREGISTRATION.md` calls it the sentinel rule H-C; that historical name maps to R-check here. Compaction H-C retains its hypothesis name and is a separate test. R-check's unchanged, changed, and inconclusive categories use the 90% Welch CI on the new minus original mean paired log ratio; a changed metric or a contradicted published class triggers a rerun, while inconclusive does not rule out change. Class contradiction means a previously equivalent outcome has a new unadjusted interval wholly outside the throughput margin, or a previously not-equivalent outcome has one wholly inside. These trigger checks are not Holm-adjusted and compare combined changes in pause, date, binary, and seeds.

The remaining measurements preserve their original descriptive status. Cells are not pooled. Median and tail trends outside the declared predictions, stall and backlog comparisons, and the fsync and hold-share accounts remain exploratory; tracing outcomes are not covariates fixed before treatment.

B. Campaign cells and auxiliary measurements

The six campaigns deliberately extend the lifetime activity covered by the initial measurements. Table VII counts diagnostics from trial envelopes, including drain. Mixed includes the nine anchor trials and had at most one flush per trial. Publication and Churn exercised reader-set publication, but the first campaign and the intervention ran no compactions because the compaction trigger was disabled. Since Lease and Snapshot differ most when reader sets change, throughput equivalence in a mostly stable set did not establish equivalence under file retirement. The compaction campaign was run to close that gap.

The intervention isolates acquisition count in eight Mixed-shaped cells: two reader counts and four non-legacy doses. It compares the default two-acquisition commit with a one-acquisition commit below the flush threshold in CoarseLock and Lease. Five rounds share a seed across each four-run cell-round unit, giving 160 trials. The default condition is rerun in the same session rather than borrowed from the first campaign. Snapshot is omitted. Section VI-B gives the predictions and exact change.

The compaction campaign uses Churn-shaped cells with $r \in \{4, 16\}$, $d \in \{0, 200, 2000\}$ μs, and compaction on or off. On uses a fixed 250 ms timer that merges all active tables into one; calls with at most one table do no work. Twelve cells, three designs, and ten rounds give 360 trials. On and off share seeds within the same reader-count, delay, and round. The implementation has a disclosed asymmetry: CoarseLock and Lease open compaction inputs under the exclusive lock, whereas Snapshot opens them outside all locks. This is not an inherent requirement of leasing.

The table-count campaign removes overlapping-table search as a factor by constructing disjoint ranges. It crosses $N \in \{4, 16, 64, 256\}$, $r \in \{4, 16\}$, and zero or one free-running writer. Sixteen cells, three designs, and ten rounds give 480 trials. The fixed 10,240-key space yields 2,560, 640, 160, or 40 keys per initial table. Delay is zero, the memtable threshold is 1 MiB, and no compaction is scheduled. A writer may still flush, so the analysis retains initial and final counts. All designs perform up to N in-memory range comparisons, under the manager lock only in CoarseLock.

Separate lifecycle microbenchmarks in the table-count results measure capture/release at one and sixteen goroutines and uncontended publication from N to $N+1$ tables. They exclude table search, memory-tier lookup, and disk I/O. Lease's test helper mirrors its scratch-copy and lease path; Snapshot uses its production pin/release path. The publication benchmark includes immediate retirement of an unpinned old Snapshot set; Lease models amortized append with spare slice capacity. Scratch is primed before timing. We report the arithmetic mean of three benchmark counts, separately from campaign throughput, without confirmatory tests.

The paced-load campaign tests whether the saturated-writer results persist at a fixed offered rate. It uses sixteen readers, one paced writer, three delay levels, two fractions of earlier Lease write capacity, and closed or open readers, with no compaction. Twelve cells over ten three-design rounds give 360 trials. The frozen rates are listed below; Section IV defines service time, response time, and censored completion tails.

The settled campaign repeats selected comparisons with a 20 s pause before every trial, after the diagnosis in Section VIII. It has nine paced cells: three delays at $f = 0.2$ with closed and open readers; 700 writes/s at 200 μs with closed and open readers, a rate chosen so earlier CoarseLock capacity was 0.78 of the offered rate; and the closed zero-delay $f = 0.8$ cell whose read comparison had been inconclusive. Ten three-design rounds give 270 trials, with commit-phase timing, a 1 ms heartbeat goroutine, and per-reader slowest-read tracking enabled. Sentinel cells copy specifications from each earlier campaign without that tracing and add 200 trials, for 470 in total, all on fresh seeds.

The Paced workload fixes $r = 16$ and uses one paced writer, four initial tables, no compaction, and a 10 s measurement window. It crosses $d \in \{0, 200, 2000\}$ μs, offered write fractions $f \in \{0.2, 0.8\}$ of earlier Lease saturated capacity, and closed or open readers. Twelve cells, three designs, and ten rounds give 360 trials. The frozen write rates at the three doses are 183/732, 155/619, and 72/286 writes/s, respectively, with each pair ordered $f = 0.2/0.8$; open-reader rates are 12,500, 8,000, 2,850 reads/s. The higher write fraction is below CoarseLock's earlier capacity at zero delay and above it at 200 and 2000 μs.

The delay probe separates nominal dose from realized lookup time. The earlier single-goroutine probe measured approximately 9, 35, and 215 μs and 2.25 ms per read; concurrent lock holds are reported separately. The sub-millisecond dose yields while spinning; the 2 ms dose sleeps before a final 0.5 ms spin.

C. Revisions, plans, runners, and seeds

The first campaign used revision `c53ee5b`, plan SHA-256 prefix `68ad96f878c1`, and runner prefix `22060ffc4885`. The intervention used `4c2749b`, plan `2e44fcacb3e7`, and runner `5dc57e7d85d7`. The compaction campaign used `c342832`, plan `5330cbddf01b`, and runner `3a05470d307b`. The table-count campaign used `adf754b`, plan `0bc0042b8b8a`, and runner `050c3210e9e8`. Protocols are `research/experiments/phase_i7/X_PREREGISTRATION.md`, `research/experiments/phase_i7/X_INTERVENTION_PREREGISTRATION.md`, `research/experiments/phase_i7/X_COMPACTION_PREREGISTRATION.md`, and `research/experiments/phase_i7/X_TABLECOUNT_PREREGISTRATION.md`.

The paced-load campaign used revision `6833d7c`; its design was frozen in `54572b2`, and the later commit changed only notes. Its plan SHA-256 prefix is `384aace771c8`, runner prefix `00f97d04191b`, and protocol `research/experiments/phase_i7/X_PACED_PREREGISTRATION.md`. The separate non-campaign append/fsync probe is recorded in

TABLE VI
FIRST-CAMPAIGN CELLS. THE LEGACY SCAN HAS NO ADDED DELAY.

Workload	Readers	Injected delay	Cells
Mixed	4, 16	0, 20, 200 μs, 2 ms, legacy scan	10
Publication	16	0, 200 μs, 2 ms, legacy scan	4
Churn	1, 4, 16	0	3

TABLE VII
WHAT THE TRIALS EXERCISED. FLUSH COUNTS INCLUDE DRAIN; COMPACTIONS COUNT COMPLETED WORK, NOT NO-OP TIMER CALLS.

Workload/campaign	Trials	Flushes/trial Mean	Flushes/trial Max	Total compactions
Mixed	309	0.55	1	0
Publication	120	20.00	20	0
Churn	90	60.19	77	0
Intervention	160	0.66	1	0
Compaction	360	36.22	87	5364
Table-count	480	0.50	1	0
Paced	360	0.17	1	0
Settled	470	7.83	61	1090

`research/experiments/phase_i7/X_READCOST/fsync_probe.py` and `research/experiments/phase_i7/X_READCOST/fsync_probe.txt`; it was not rerun for this paper update.

The settled campaign's design was frozen at revision `54b2c67`, with plan SHA-256 prefix `7c868047870d`, runner prefix `35b8ea6bbc3a`, and protocol `research/experiments/phase_i7/X_SETTLED_PREREGISTRATION.md`. Exploratory evidence, excluded from every campaign analysis, is in `research/experiments/phase_i7/X_READCOST/stall_diag/` and `research/experiments/phase_i7/X_READCOST/fsync_checks/`, with the narrative in `research/experiments/phase_i7/X_READCOST/OVERNIGHT_NOTES.md`. The diagnostic tool's results carry no git revision; the fsync checks used a binary built at `f71e7e2` with SHA-256 prefix `d0e7bb68b0a8`.

Master seeds were 20261030, 20261103, 20261104, 20261105, 20261106, and 20261107, respectively. Paired seeds are the first eight SHA-256 bytes of decimal master, cell or shared base identifier, and round separated by vertical bars, interpreted as a big-endian signed 64-bit value; compaction on/off shares the base identifier. Plans and ledgers retain execution order, retry and approval bindings, and the excluded smoke runs; the first start stopped after 38 successful trials is disclosed in the protocol.

APPENDIX B
FULL RESULTS

Full cells, uncertainty intervals, and auxiliary results retain their original data captions. Figures keep the original confidence intervals; table macros copy the existing generated results.

Retirement ran in all on cells in the exploratory lifecycle measurements. Mean retirement lags spanned 0.74–1.26 ms for CoarseLock, 0.86–6.21 ms for Lease, and 0.96–3.81 ms for Snapshot. Cell mean retired-table high-water marks were 4.0–5.9 across the designs (Table XIV). At low doses, counts were roughly timer-limited, with cell means of 32–36 compactions per trial. At 2 ms they were flush-limited: CoarseLock completed mean counts of 9–10 versus 22.0–22.8 in the alternatives because it produced fewer flushes. These observations do not establish compaction starvation.

The exploratory per-table critical-section increase was roughly 12 ns at four readers and 32 ns at sixteen, compared with about 3.8 ns per table in the single-goroutine microbenchmark. Cold cache lines on real table objects spread through the heap are a possible explanation, not a tested one. The microbenchmark uses synthetic table objects and isolates lifecycle operations; it is not an end-to-end read measurement.

The higher offered fraction was not a comfortable operating point for any design in the exploratory descriptive comparison. For Lease and Snapshot, cell medians of trial write response p99 ranged from 65–453 ms, while cell medians of trial p50 were 1.0–3.0 ms. Even trials above the 0.95 completion flag left up to 360 ticks unissued, so the flag understates censoring. The descriptive Lease/Snapshot completion-p99 ratios and wide 95% CIs are reported in Table XVIII; no prediction or equivalence conclusion applies to them. Their write service p50 ratios stayed at 0.999–1.004. Similar service medians do not establish similar response tails.

APPENDIX C
PROJECT HISTORY AND EXECUTION NOTES

A. Earlier measurements

A defective reader motivated the present study but supplies no pooled evidence for it. That reader scanned from the target block toward the data end, used about four unbuffered syscalls per record, and did not stop after passing the target key. A 960-trial WSL2 campaign comparing CoarseLock and Snapshot and a Windows campaign stopped by its stopping rule at 60 of 330 planned trials, with 18 complete triplets, used this path. They reported Snapshot completing 3.3–18.5 times CoarseLock's writes and churn throughput ratios of 0.63–0.74 at $r \leq 4$. We retain the defective path only as the legacy scan operating point. The earlier campaigns motivated the controlled delay, were not pooled with these data, and carry no inference into this paper. This study supersedes their claims, including the withdrawn $\pm 5\%$ equivalence claim and any earlier claim about Lease under churn.

B. Smoke trials and the aborted start

Smoke trials were seen before each freeze and excluded. The first campaign had 21 smoke trials before analysis rules were fixed; the intervention had eight after predictions were written; compaction had fifteen; table count had six; paced load had six after predictions were written. The settled campaign had six mechanical smoke trials before its freeze, from a dirty tree with an earlier runner build, with outcomes unread, and 36 non-campaign calibration trials whose outcomes were seen. After seeing that calibration and before any campaign

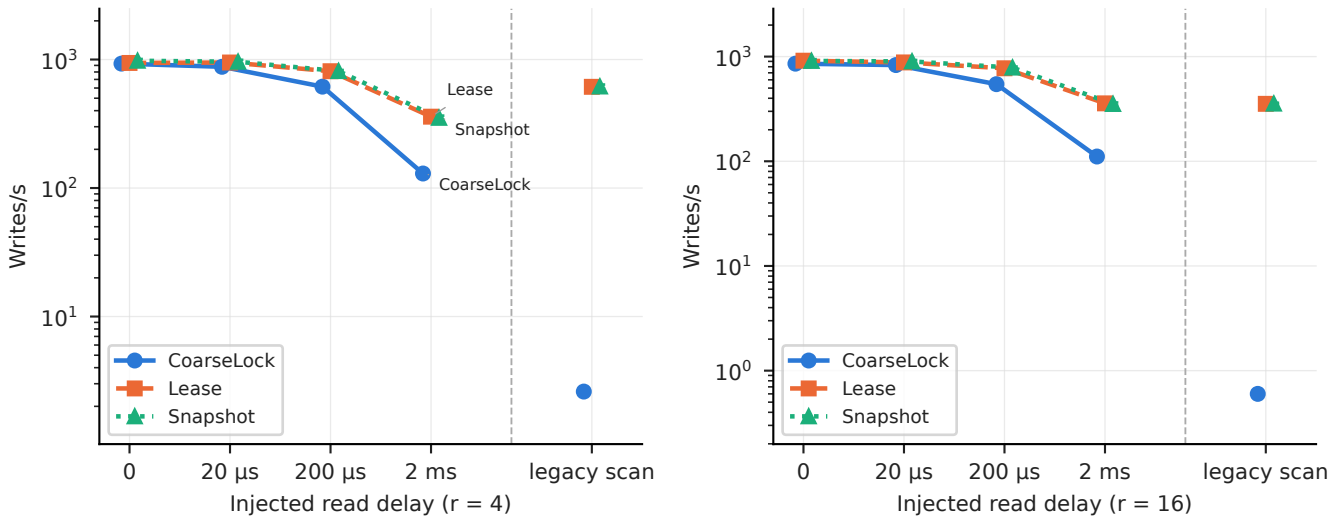


Fig. 9. Mixed workload writes, ten triplets per cell. Points are per-design geometric means; bars are exponentiated 95% log-t CIs. Most intervals are smaller than the markers. The legacy scan is an isolated category.

TABLE VIII
MIXED WORKLOAD, TEN TRIPLETS PER CELL. RATES ARE GEOMETRIC MEANS (OPERATIONS/S); HOLD IS THE ARITHMETIC MEAN OF COARSELOCK'S TRIAL MEAN READ CRITICAL SECTIONS. PAIRED RATIOS HAVE 95% t CIS.

		Hold	Writes/s			S	Reads/s			R_{CL}/R_L
r	Delay	μs	CoarseLock	Lease	Snapshot	[95% CI]	CoarseLock	Lease	Snapshot	[95% CI]
4	0	13.6	929	940	978	0.988 [0.896, 1.089]	7,468	6,258	6,482	1.193 [1.089, 1.308]
4	20 μs	33.6	878	948	961	0.926 [0.881, 0.974]	7,035	4,844	4,909	1.452 [1.380, 1.528]
4	200 μs	163	615	811	813	0.758 [0.725, 0.793]	4,932	4,154	4,161	1.187 [1.136, 1.241]
4	2 ms	1,885	130	359	352	0.361 [0.354, 0.368]	1,039	1,434	1,408	0.725 [0.710, 0.739]
4	legacy scan	124,853	2.6	613	617	0.004250 [0.004049, 0.004461]	21.2	43.3	43.3	0.489 [0.468, 0.511]
16	0	21.2	856	914	909	0.937 [0.884, 0.993]	27,538	25,101	24,972	1.097 [1.041, 1.156]
16	20 μs	39.3	830	880	899	0.944 [0.887, 1.004]	26,668	18,620	18,966	1.432 [1.350, 1.519]
16	200 μs	173	545	773	785	0.705 [0.676, 0.736]	17,507	16,017	16,256	1.093 [1.048, 1.140]
16	2 ms	1,915	111	358	352	0.311 [0.305, 0.316]	3,557	5,693	5,617	0.625 [0.614, 0.636]
16	legacy scan	430,130	0.6	354	355	0.001696 [0.001661, 0.001732]	21.2	40.6	40.7	0.522 [0.501, 0.544]

trial, we raised the H-S threshold from 25 to 40 ms. The table-count single-round smoke ratio near 0.66 at 256 tables and sixteen readers pointed in the predicted direction, although predictions were written first. No design value, margin, or prediction changed because of these runs. An operator also stopped the first start at 38 of 519 trials for power-supply reasons unrelated to results. It was discarded without analysis; only status lines were seen. Fixtures were constructed without fsync.

C. Power record and settled execution deviations

`platform.json` records the Balanced base plan. The overnight campaigns ran on AC power with other user applications closed by the operator; the Best performance overlay on the Balanced base plan was read back from the registry after both had finished, not during them. This does not independently verify the overlay throughout execution. Defender and other host activity remain possible sources of variation. The first five campaigns ran back to back, so each trial could inherit host effects from the one before. The settled sentinels bound this only in the rechecked cells, and they confound the pause with date, binary, and seeds. Bursts near 31 ms were not controlled in any campaign. During the settled campaign a helper process held a power request to prevent idle standby and spawned about 50 short `tasklist` processes; one read-only status check overlapped one trial, which is the largest of its group and is retained under the no-exclusion rule, though no prediction depends on its tail; the desktop application was also running, and the operator was away, so it cannot be confirmed that the machine stayed untouched.

The settled pause and H-S threshold were selected after non-campaign calibration. Its 36 seen trials were excluded from campaign inference, and its six mechanical smoke trials used a dirty tree and an earlier runner build with outcomes unread. No campaign trial preceded the change from the draft 25 ms H-S threshold to 40 ms. The overlapping status check, helper activity, and desktop application described above remain execution deviations, with the affected trial retained under the no-exclusion rule.

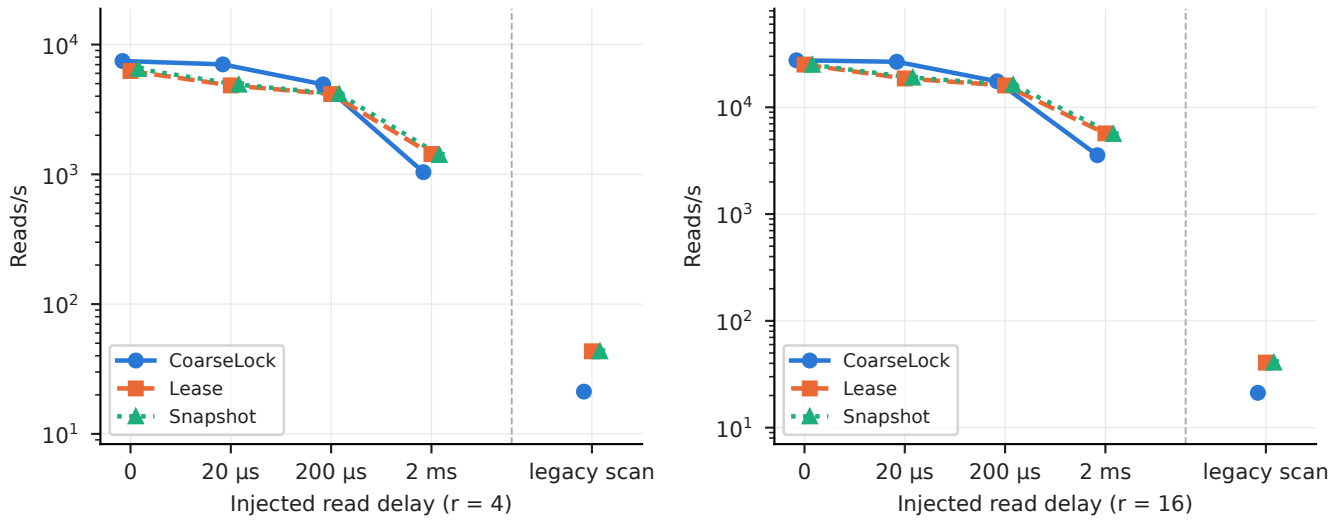


Fig. 10. Mixed workload reads, ten triplets per cell. Points are geometric means; bars are exponentiated 95% log-t CIs. CoarseLock's low-dose read gain was unpredicted.

TABLE IX
INTERVENTION. q IS ARITHMETIC; THROUGHPUT RATIOS ARE PAIRED GEOMETRIC MEANS, WITH 95% PAIRED LOG-t CIS WHERE SHOWN. COLUMN NUMBERS 2 AND 1 DENOTE TWO-ACQUISITION AND ONE-ACQUISITION COMMITS.

r	Delay	q_{CL} 2	q_{CL} 1	q_L 2	q_L 1	$W_{CL,1}/W_{CL,2}$ [95% CI]	S 2	S 1	R_{CL}/R_L [95% CI] 2	R_{CL}/R_L [95% CI] 1
4	0	2.008	1.004	1.655	1.004	1.014 [0.943, 1.091]	0.965	0.963	1.170 [1.066, 1.285]	0.963 [0.897, 1.033]
4	20 µs	2.003	1.002	1.281	1.002	1.032 [0.928, 1.147]	0.946	0.974	1.479 [1.363, 1.604]	0.975 [0.882, 1.077]
4	200 µs	2.003	1.002	1.281	1.002	1.191 [1.131, 1.253]	0.731	0.864	1.144 [1.062, 1.232]	0.864 [0.758, 0.985]
4	2 ms	2.003	1.001	0.998	0.857	1.468 [1.429, 1.508]	0.356	0.539	0.715 [0.706, 0.726]	0.630 [0.592, 0.671]
16	0	2.010	1.003	1.722	1.003	1.091 [1.050, 1.134]	0.945	0.987	1.103 [1.064, 1.142]	0.987 [0.895, 1.089]
16	20 µs	2.008	1.003	1.320	1.003	1.123 [0.992, 1.271]	0.855	0.983	1.301 [1.174, 1.443]	0.983 [0.897, 1.077]
16	200 µs	2.006	1.001	1.290	1.001	1.274 [1.175, 1.382]	0.662	0.848	1.030 [0.993, 1.068]	0.848 [0.822, 0.875]
16	2 ms	2.003	1.001	0.997	0.858	1.528 [1.507, 1.548]	0.310	0.479	0.623 [0.603, 0.643]	0.559 [0.553, 0.566]

TABLE X
PUBLICATION WORKLOAD. RATES AND TRIAL LOCK-WAIT P99 (µS) ARE GEOMETRIC MEANS. PUBLICATION COUNTS ARE MEDIAN [MIN, MAX], OUT OF TWENTY.

	Reads/s			In-window publications			Lock-wait p99 (µs)		
Delay	CoarseLock	Lease	Snapshot	CoarseLock	Lease	Snapshot	CoarseLock	Lease	Snapshot
0	318,567	304,721	314,201	20 [20, 20]	20 [20, 20]	20 [20, 20]	247	41.60	13.37
200 µs	75,018	74,678	74,844	20 [20, 20]	20 [20, 20]	20 [20, 20]	450	19.33	12.10
2 ms	8,437	8,445	8,450	20 [20, 20]	20 [20, 20]	20 [20, 20]	3,886	1.51	2.83
legacy scan	27.6	42.7	42.7	10.5 [9, 11]	20 [20, 20]	20 [20, 20]	1,054,655	1.88	1.74

TABLE XI
PUBLICATION LOCK-WAIT P99: PAIRED LEASE/SNAPSHOT RATIO, 90% t CI, AND PLANNED CLASSIFICATION.

Delay	Ratio [90% CI]	Classification
0	3.11 [1.54, 6.30]	not equivalent
200 µs	1.60 [0.51, 4.98]	inconclusive
2 ms	0.53 [0.22, 1.29]	inconclusive
legacy scan	1.08 [0.64, 1.82]	inconclusive

D. Race-detector context

The race detector reported no data races in the engine and campaign harness tests under Linux on WSL2, rerun after commit-phase tracing was added; one older overload test fails under the detector for a timing reason because the slowdown disturbs the overload it expects. The older timing-sensitive failure is the E4 overload test. This was not a Windows race-detector run.

The older `i7trial` tool, unused by these campaigns, does not build on Linux.

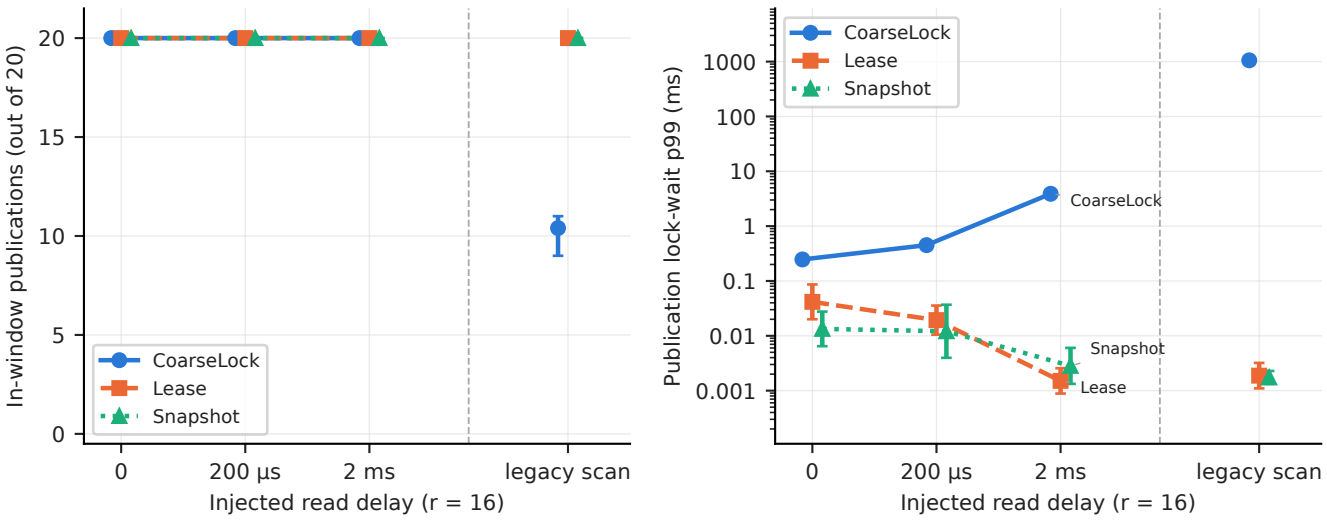


Fig. 11. Publication workload, ten triplets per cell. Left: arithmetic mean in-window count with min–max bars. Right: geometric mean trial publication lock-wait p99 and exponentiated 95% log-t CIs. Wait counters include drain.

TABLE XII

CHURN, TEN TRIPLETS PER CELL. RATES ARE GEOMETRIC MEANS (OPERATIONS/S). LEASE/SNAPSHOT HAS 95% PAIRED-t AND PERCENTILE BOOTSTRAP CIS. FLUSHES ARE ARITHMETIC MEANS INCLUDING DRAIN, ORDERED COARSELOCK/LEASE/SNAPSHOT.

r	Outcome	CoarseLock	Lease	Snapshot	Ratio [95% t CI]	Bootstrap CI	Flushes
1	W	697	747	781	0.957 [0.905, 1.012]	[0.910, 0.997]	65.3/70.1/73.2
1	R	1,400	807	844	0.956 [0.904, 1.011]	[0.909, 0.997]	–
4	W	585	702	699	1.004 [0.963, 1.047]	[0.970, 1.038]	54.6/65.9/65.5
4	R	4,703	3,089	3,074	1.005 [0.964, 1.048]	[0.970, 1.038]	–
16	W	415	569	588	0.968 [0.933, 1.004]	[0.938, 0.995]	38.8/53.2/55.1
16	R	13,351	10,050	10,357	0.970 [0.937, 1.005]	[0.941, 0.997]	–

TABLE XIII

COMPACTION-ON THROUGHPUT. LEASE/SNAPSHOT RATIOS HAVE 90% CIS AND HOLM-ADJUSTED CLASSIFICATIONS; S AND COARSELOCK/LEASE READ RATIOS HAVE 95% CIS. ALL RATIOS ARE PAIRED GEOMETRIC MEANS.

		Lease/Snapshot writes		Lease/Snapshot reads		S	R_{CL}/R_L
r	Delay	Ratio [90% CI]	Class	Ratio [90% CI]	Class	[95% CI]	[95% CI]
4	0	0.989 [0.944, 1.035]	equivalent	0.996 [0.954, 1.040]	equivalent	0.919 [0.881, 0.958]	1.494 [1.433, 1.557]
4	200 μs	0.997 [0.949, 1.047]	equivalent	0.997 [0.949, 1.047]	equivalent	0.681 [0.651, 0.712]	1.301 [1.244, 1.361]
4	2 ms	0.995 [0.975, 1.015]	equivalent	0.987 [0.968, 1.008]	equivalent	0.439 [0.430, 0.448]	0.912 [0.892, 0.933]
16	0	1.030 [0.980, 1.083]	equivalent	1.038 [0.989, 1.090]	equivalent	0.851 [0.800, 0.904]	1.408 [1.326, 1.496]
16	200 μs	0.949 [0.909, 0.991]	equivalent	0.973 [0.933, 1.015]	equivalent	0.656 [0.636, 0.676]	1.239 [1.204, 1.276]
16	2 ms	0.998 [0.980, 1.016]	equivalent	0.990 [0.972, 1.007]	equivalent	0.393 [0.386, 0.400]	0.820 [0.806, 0.835]

TABLE XIV

COMPACTION-ON LIFECYCLE MEASUREMENTS. ARITHMETIC MEANS OF TRIAL COMMIT-WAIT P99 AND TRIAL MEAN RETIREMENT LAG, INCLUDING DRAIN.

		Commit-wait p99 (μs)			Mean retirement lag (ms)		
r	Delay	CoarseLock	Lease	Snapshot	CoarseLock	Lease	Snapshot
4	0	305.8	49.5	29.2	0.85	0.86	0.96
4	200 μs	1038.3	21.5	11.7	0.78	1.52	1.29
4	2 ms	7795.5	70.6	12.8	0.76	5.11	2.86
16	0	627.9	147.5	75.4	1.26	1.02	0.97
16	200 μs	1660.5	245.6	176.7	0.85	1.73	1.47
16	2 ms	9220.9	132.4	16.8	0.74	6.21	3.81

TABLE XV

TABLE-COUNT CAMPAIGN, TEN TRIPLETS PER CELL. THROUGHPUT RATIOS ARE LEASE/SNAPSHOT WITH 90% PAIRED LOG-t CIS AND PLANNED HOLM-ADJUSTED CLASSES. CS IS ARITHMETIC MEAN READ CRITICAL SECTION (μS). A DASH MEANS NO WRITER AND NO APPLICABLE WRITE TEST.

			Reads		Writes		Mean read CS (μs)		
N	r	Writers	Ratio [90% CI]	Class	Ratio [90% CI]	Class	CoarseLock	Lease	Snapshot
4	4	0	0.975 [0.958, 0.993]	equivalent	–	–	13.520	0.326	0.256
4	4	1	0.989 [0.963, 1.016]	equivalent	0.988 [0.960, 1.018]	equivalent	13.546	2.274	2.109
4	16	0	0.989 [0.981, 0.997]	equivalent	–	–	40.980	0.393	0.327
4	16	1	0.982 [0.960, 1.005]	equivalent	0.980 [0.953, 1.007]	equivalent	21.682	1.708	1.542
16	4	0	0.956 [0.944, 0.969]	equivalent	–	–	10.524	0.494	0.294
16	4	1	0.947 [0.920, 0.976]	equivalent	0.943 [0.912, 0.974]	equivalent	13.792	2.304	2.095
16	16	0	0.962 [0.924, 1.002]	equivalent	–	–	26.098	0.914	0.610
16	16	1	0.981 [0.940, 1.024]	equivalent	0.977 [0.927, 1.031]	equivalent	19.559	1.768	1.556
64	4	0	0.918 [0.906, 0.929]	equivalent	–	–	10.250	1.016	0.291
64	4	1	1.037 [0.997, 1.078]	equivalent	1.045 [1.001, 1.092]	equivalent	14.962	2.558	2.117
64	16	0	0.851 [0.837, 0.866]	not equivalent	–	–	28.984	2.689	0.802
64	16	1	0.978 [0.943, 1.013]	equivalent	0.979 [0.936, 1.023]	equivalent	20.045	2.175	1.609
256	4	0	0.712 [0.699, 0.726]	not equivalent	–	–	12.157	3.370	0.269
256	4	1	0.994 [0.954, 1.036]	equivalent	1.011 [0.967, 1.057]	equivalent	17.045	3.528	2.184
256	16	0	0.663 [0.660, 0.666]	not equivalent	–	–	33.703	8.512	0.877
256	16	1	0.972 [0.933, 1.012]	equivalent	0.985 [0.938, 1.035]	equivalent	22.349	3.900	1.602

TABLE XVI

SEPARATE LIFECYCLE MICROBENCHMARKS: ARITHMETIC MEAN OF THREE COUNTS, NS/OPERATION. READ-CYCLE BENCHMARKS ALLOCATE NO MEMORY. PUBLICATION ALLOCATION COUNTS ARE PER OPERATION.

	Read cycle, 1 goroutine		Read cycle, 16 goroutines		Publication		Publication allocations	
N	Lease	Snapshot	Lease	Snapshot	Lease	Snapshot	Lease	Snapshot
4	26.7	21.0	58.1	88.6	109.1	653.7	0	3
16	69.6	20.8	137.5	88.0	104.4	717.5	0	3
64	248.1	20.9	481.8	88.1	110.2	1035.3	0	3
256	974.4	20.9	1766.7	87.6	127.7	3200.3	0	3

TABLE XVII

PACED WORKLOAD, TEN TRIPLETS PER CELL. ACHIEVED FRACTIONS AND WRITE RESPONSE P50 ARE ARITHMETIC MEANS OF TRIAL SUMMARIES. SATURATED COUNTS ARE COARSELOCK TRIALS BELOW 0.95 COMPLETED/OFFERED. THROUGHPUT IS THE PAIRED GEOMETRIC COARSELOCK/LEASE RATIO WITH ITS FROZEN 90% CI AND HOLM-ADJUSTED H-P2 OR H-P3 CLASS. ROUNDED 1.000 INTERVALS ARE NOT EXACT EQUALITY.

			Offered	Achieved write fraction			Saturated	CoarseLock/Lease writes		Response p50 (ms)	
Delay	f	Readers	writes/s	CoarseLock	Lease	Snapshot	CoarseLock	Ratio [90% CI]	Class	CoarseLock	Lease
0	0.2	closed	183	0.999	0.999	0.999	0	1.000 [1.000, 1.000]	equivalent	1.232	1.126
0	0.2	open	183	0.999	0.999	0.999	0	1.000 [1.000, 1.000]	equivalent	1.127	1.119
0	0.8	closed	732	0.995	0.989	0.995	0	1.007 [0.995, 1.019]	equivalent	1.322	7.023
0	0.8	open	732	0.989	0.991	0.997	0	0.998 [0.982, 1.015]	equivalent	29.135	22.340
200 μs	0.2	closed	155	0.998	0.999	0.999	0	0.999 [0.997, 1.001]	equivalent	1.888	1.272
200 μs	0.2	open	155	0.998	0.999	0.999	0	0.999 [0.996, 1.001]	equivalent	1.663	1.244
200 μs	0.8	closed	619	0.884	0.986	0.994	10	0.896 [0.866, 0.927]	inconclusive	631.452	1.160
200 μs	0.8	open	619	0.912	0.991	0.999	9	0.920 [0.898, 0.942]	inconclusive	457.808	2.169
2 ms	0.2	closed	72	0.999	0.999	0.999	0	1.000 [1.000, 1.000]	equivalent	8.084	3.015
2 ms	0.2	open	72	0.998	0.999	0.999	0	0.999 [0.998, 1.001]	equivalent	7.504	3.057
2 ms	0.8	closed	286	0.388	0.995	0.999	10	0.390 [0.387, 0.392]	not equivalent	3085.330	2.966
2 ms	0.8	open	286	0.404	0.996	1.000	10	0.405 [0.401, 0.409]	not equivalent	3056.809	3.085

TABLE XVIII

PACED DESCRIPTIVE RATIOS, WITH FROZEN 95% PAIRED LOG-t CIS. ALL COLUMNS EXCEPT THE LAST COMPARE COARSELOCK/LEASE. THE READ COLUMN IS THROUGHPUT FOR CLOSED READERS AND RESPONSE P99 FOR OPEN READERS. THE LAST COLUMN COMPARES LEASE/SNAPSHOT WRITE RESPONSE COMPLETION-P99, WITH NO PREDICTION; UNISSUED TICKS HAVE NO OBSERVED LATENCY, AND CENSORED RATIOS DO NOT BOUND TRUE RATIOS. NO MEDIAN OR TAIL TREND TEST IS ADDED.

			CoarseLock/Lease writes		CoarseLock/Lease	Lease/Snapshot
Delay	f	Readers	Response p50	Service p50	Read metric	Response p99
0	0.2	closed	1.093 [1.062, 1.124]	1.088 [1.064, 1.113]	0.967 [0.912, 1.025]	0.957 [0.658, 1.393]
0	0.2	open	1.007 [0.995, 1.020]	1.008 [0.994, 1.022]	1.975 [0.892, 4.372]	0.748 [0.326, 1.720]
0	0.8	closed	0.847 [0.319, 2.251]	1.068 [1.057, 1.080]	0.776 [0.666, 0.904]	0.761 [0.544, 1.067]
0	0.8	open	0.879 [0.135, 5.749]	1.009 [1.003, 1.015]	1.086 [0.521, 2.261]	0.880 [0.596, 1.300]
200 μs	0.2	closed	1.484 [1.473, 1.496]	1.472 [1.461, 1.482]	0.921 [0.889, 0.954]	0.868 [0.166, 4.551]
200 μs	0.2	open	1.336 [1.324, 1.349]	1.342 [1.330, 1.354]	1.151 [0.272, 4.862]	0.588 [0.167, 2.064]
200 μs	0.8	closed	510.629 [395.760, 658.837]	1.562 [1.552, 1.572]	0.550 [0.491, 0.616]	2.173 [0.941, 5.018]
200 μs	0.8	open	279.497 [155.636, 501.930]	1.416 [1.406, 1.427]	0.734 [0.288, 1.874]	1.141 [0.449, 2.897]
2 ms	0.2	closed	2.681 [2.659, 2.703]	2.677 [2.657, 2.697]	0.720 [0.711, 0.730]	1.386 [0.839, 2.288]
2 ms	0.2	open	2.453 [2.388, 2.520]	2.447 [2.385, 2.510]	4.712 [3.516, 6.315]	0.702 [0.402, 1.225]
2 ms	0.8	closed	1040.347 [1028.850, 1051.972]	3.104 [3.101, 3.108]	0.642 [0.632, 0.653]	0.914 [0.361, 2.317]
2 ms	0.8	open	992.262 [945.816, 1040.989]	3.035 [3.022, 3.048]	1.278 [0.626, 2.609]	1.129 [0.306, 4.170]

TABLE XIX

FSYNCS OF 25 MS OR LONGER, BY WRITE RATE. SETTLED TRIALS ARE ALL THREE DESIGNS. BARE RUNS ARE EXPLORATORY, WITH NO ENGINE. "NO FIRST" EXCLUDES EACH RUN'S FIRST FSYNC. THE LAST ROW'S BARE RUNS WERE AT 700 WRITES/S.

	Settled		Bare		
Writes/s	Trials	Fsyncs (%)	Runs	No first	Fsyncs (%)
72	6/60	0.044	0/10	0/10	0.000
155	16/60	0.140	2/10	0/10	0.013
183	28/60	0.229	2/10	1/10	0.093
650–730	88/90	0.283	5/10	3/10	0.053

TABLE XX

EXPLORATORY FSYNC ON/OFF CHECK AT ZERO DELAY, 732 WRITES/S, CLOSED READERS, TEN ROUNDS. RATIOS ARE PAIRED GEOMETRIC LEASE/SNAPSHOT MEANS WITH 90% CIS. CLASSES USE THE DECLARED MARGIN WITH NO MULTIPLICITY CORRECTION. ROUNDED 1.000 INTERVALS ARE NOT EXACT EQUALITY.

	Reads/s		Trials with fsync	Hold	Lease/Snapshot reads		Lease/Snapshot writes	
Fsync	Lease	Snapshot	≥ 25 ms	(%)	Ratio [90% CI]	Class	Ratio [90% CI]	Class
fsync on	108,009–173,846	104,776–156,826	17/20	72.3	1.052 [0.906, 1.222]	inconclusive	1.011 [0.989, 1.034]	equivalent
fsync off	437,665–454,848	427,814–460,161	0/20	4.3	0.994 [0.987, 1.000]	equivalent	1.000 [1.000, 1.000]	equivalent